



Plataforma de Pre-Explotación y Análisis Ofensivo de
Binarios

Desarrollo de Aplicaciones Multiplataforma / Presencial

Mario Asenjo Pérez

Antonio Reinoso Peinado

ÍNDICE

ABSTRACT (Español)	5
ABSTRACT (Inglés)	6
1. JUSTIFICACIÓN DEL PROYECTO	7
1.1 Motivación del proyecto	7
1.2 Estado de la cuestión	7
1.3 Público objetivo	8
1.4 Comparativa razonada	9
1.5 Valor añadido del proyecto	11
2. INTRODUCCIÓN	12
2.1 Principales requisitos del proyecto:	12
3. OBJETIVOS	14
3.1 R01 – El programa debe permitir acceder solo a usuarios autorizados	14
3.2 R02 – El programa debe poder guardar datos y mantener historiales de análisis	15
3.3 R03 – El programa debe permitir subir binarios para su análisis	15
3.4 R04 – El programa debe analizar el binario de forma estática (ELF MVP)	16
3.5 R05 – El programa debe ejecutar un análisis dinámico controlado	17
3.6 R06 – El programa debe guardar y mostrar resultados del análisis	17
3.7 R07 – El programa debe generar un informe del análisis	17
3.8 R08 – El programa debe registrar quién realiza cada acción (auditoría)	18
4. DESCRIPCIÓN	19
4.1 Arquitectura de la solución	19
4.2 Casos de uso de la aplicación	20
5. DISEÑOS	36
5.1 Diagrama de clases (Backend):	36
5.2 Diagrama de clases (Worker Static):	36
5.3 Diagrama de clases (Worker Dynamic):	37
5.4 Diagrama E/R	37

5.5 Diagrama de la base de datos.....	38
5.6 Diagrama de flujo de navegación.....	39
5.7 Interfaces.....	40
6. TECNOLOGÍA.....	47
7. METODOLOGÍA.....	52
7.1 Enfoque metodológico.....	52
7.2 Justificación de la metodología.....	52
7.3 Fases de trabajo.....	53
7.4 Planificación inicial por requisitos.....	54
7.5 Planificación final y desviaciones.....	55
7.6 Validación metodológica.....	56
7.7 Diagrama de Gantt.....	57
8. PRESUPUESTO.....	58
8.1 Coste de Desarrollo por Requisito.....	58
8.2 Costes de infraestructura y herramientas.....	59
8.3 Coste Total Estimado.....	60
9. README y GIT.....	60
10. TRABAJOS FUTUROS.....	61
11. CONCLUSIONES.....	62
12. REFERENCIAS.....	63

ABSTRACT (Español)

La fase de pre-explotación constituye un paso esencial en una auditoría de seguridad ofensiva, ya que permite identificar y comprender la superficie de ataque de una aplicación antes de intentar cualquier explotación. Sin embargo, en muchos entornos reales esta fase se apoya en herramientas aisladas y procesos manuales que generan información dispersa y difícil de correlacionar.

Este proyecto propone el desarrollo de una plataforma de pre-explotación y análisis de binarios, orientada a centralizar y estructurar el análisis inicial de ejecutables, con especial foco en binarios ELF sobre sistemas Linux. La solución combina análisis estático y análisis dinámico controlado, permitiendo obtener información relevante sobre la estructura del binario, sus mitigaciones de seguridad y su comportamiento durante la ejecución.

La plataforma se concibe como un producto extensible, basado en una arquitectura desacoplada que integra un backend desarrollado en Java mediante Spring Boot, una interfaz web y workers especializados para la ejecución de los análisis. Los resultados se almacenan de forma persistente utilizando PostgreSQL con soporte JSONB, facilitando la trazabilidad y el análisis posterior de las evidencias obtenidas.

Asimismo, el sistema incorpora mecanismos de autenticación, control de permisos y auditoría, permitiendo registrar de forma trazable quién realiza cada acción y cuándo. A partir de los datos recopilados, se aplica un sistema de puntuación de riesgos que facilita la priorización de hallazgos y apoya la toma de decisiones técnicas en fases posteriores de la auditoría. El proyecto adopta un enfoque ético, excluyendo funcionalidades de explotación automática y orientándose a entornos controlados de evaluación de seguridad.

ABSTRACT (Inglés)

The pre-exploitation phase is a fundamental step in offensive security audits, as it allows the identification and understanding of an application's attack surface before any exploitation attempts are made. However, in many real-world environments, this phase relies on isolated tools and manual processes that produce fragmented and hard-to-correlate information.

This project presents the development of a pre-exploitation platform for binary analysis, focused on ELF binaries running on Linux systems. The solution combines static analysis and controlled dynamic analysis to obtain relevant information about the binary structure, security mitigations, and runtime behavior.

The platform is conceived as an extensible product, based on a decoupled architecture that includes a backend implemented in Java using Spring Boot, a web-based interface, and specialized workers responsible for executing the analysis tasks. Results are persistently stored in PostgreSQL using JSONB, enabling traceability and further inspection of collected evidence.

In addition, the system incorporates authentication, authorization, and auditing mechanisms to record who performs each action and when. Based on the collected data, a risk scoring system is applied to prioritize findings and support technical decision-making in later stages of the audit. The project follows an ethical approach, explicitly excluding automatic exploitation features and targeting controlled security assessment environments.

1. JUSTIFICACIÓN DEL PROYECTO

1.1 Motivación del proyecto

La motivación principal de este proyecto surge del interés por la seguridad ofensiva y por la necesidad de estructurar de forma adecuada la fase de pre-explotación dentro de una auditoría de seguridad. Esta fase resulta clave para comprender la superficie de ataque de un binario, evaluar sus mecanismos de protección y orientar correctamente los esfuerzos posteriores de análisis técnico.

En la práctica profesional, esta etapa suele apoyarse en un conjunto de utilidades independientes y en interpretación manual de resultados, lo que fragmenta la información y complica su consolidación y trazabilidad. Además, la ausencia de mecanismos de priorización obliga a depender en exceso del criterio individual del auditor, incrementando el riesgo de errores o análisis incompletos.

Este proyecto nace con el objetivo de ordenar y sistematizar esta fase, proporcionando una solución que centralice el análisis inicial de binarios y facilite una visión global, reproducible y priorizada de los resultados obtenidos.

1.2 Estado de la cuestión

Actualmente existen diversas herramientas ampliamente utilizadas en auditorías ofensivas para el análisis de binarios, como utilidades de análisis estático, herramientas de inspección de mitigaciones de seguridad o sistemas de trazado dinámico. Aunque estas herramientas son muy potentes de forma individual, presentan una serie de limitaciones comunes:

- Están diseñadas para resolver problemas concretos y no como soluciones integradas.
- Generan salidas heterogéneas que requieren interpretación manual.

- No ofrecen mecanismos nativos de correlación ni de priorización de riesgos.
- Carecen de trazabilidad sobre quién ha realizado el análisis y en qué contexto.
- No están pensadas para flujos de trabajo multiusuario.

Por otro lado, existen plataformas comerciales de análisis de malware o sandboxing que sí proporcionan soluciones más integradas, pero suelen ser cerradas, complejas de desplegar o excesivas para escenarios de auditoría técnica, laboratorio o formación avanzada.

Ante este contexto, se identifica un espacio claro para una solución intermedia: una plataforma abierta, extensible y orientada específicamente a la pre-explotación, que integre análisis estático y dinámico sin llegar a la explotación automática.

1.3 Público objetivo

El proyecto está dirigido principalmente a:

- Profesionales y estudiantes avanzados de ciberseguridad ofensiva.
- Auditores de seguridad que necesiten estructurar la fase inicial de análisis.
- Equipos técnicos que trabajen en entornos de laboratorio o evaluación controlada.
- Contextos académicos donde se requiera trazabilidad y reproducibilidad del análisis.

La plataforma no está orientada a usuarios sin conocimientos técnicos previos, sino a perfiles que ya comprenden los fundamentos del análisis de binarios y desean mejorar su eficiencia y organización.

1.4 Comparativa razonada

Criterio	Herramientas aisladas	Plataformas comerciales cerradas	PPAOB
Integración de resultados	Baja. Cada herramienta genera su propia salida.	Alta, pero dependiente de la plataforma.	Alta dentro del flujo definido por el proyecto.
Análisis estático	Disponible mediante herramientas específicas.	Normalmente incluido.	Incluido para binarios ELF como parte del MVP.
Análisis dinámico	Posible con herramientas separadas.	Normalmente incluido en sandbox.	Incluido de forma controlada mediante worker dinámico y agente C.
Correlación de evidencias	Manual o inexistente.	Automatizada, pero poco personalizable.	Automatizada mediante señales, scoring y priorización.
Priorización de hallazgos	Depende del criterio manual del auditor.	Incluida, pero ligada a reglas internas de la solución.	Incluida mediante scoring 0..100, donde mayor puntuación implica mayor criticidad.
Trazabilidad de acciones	Limitada o inexistente.	Habitualmente disponible.	Disponible mediante auditoría append-only asociada a usuarios y acciones.
Persistencia histórica	Manual o dependiente de ficheros externos.	Integrada.	Integrada mediante PostgreSQL y JSONB.
Gestión de binarios y artefactos	Manual.	Integrada.	Integrada mediante MinIO/S3 y referencias en base de datos.

Criterio	Herramientas aisladas	Plataformas comerciales cerradas	PPAOB
Multiusuario y roles	No suele estar presente.	Sí, normalmente.	Sí, mediante autenticación JWT y RBAC.
Generación de informes	Manual o externa.	Integrada.	Integrada mediante informes HTML generados desde los resultados.
Flexibilidad técnica	Alta, pero poco estructurada.	Limitada por la plataforma.	Alta, al ser una solución propia y modular.
Coste económico	Bajo, si se usan herramientas libres.	Alto o dependiente de licencia.	Bajo en entorno académico, usando tecnologías abiertas.
Adecuación académica	Parcial, porque exige mucha integración manual.	Menor, por ser cerrada y difícil de justificar internamente.	Alta, porque permite demostrar arquitectura, desarrollo, persistencia, seguridad y análisis técnico.
Riesgo de sobrealcance	Bajo en herramientas aisladas, pero con poca visión de producto.	Alto por complejidad e infraestructura.	Controlado mediante un MVP definido y ampliaciones futuras.

Tabla 1: comparativa razonada con herramientas existentes

A partir de esta comparación, se puede concluir que PPAOB aporta valor principalmente en tres aspectos. En primer lugar, centraliza información que normalmente estaría dispersa entre diferentes herramientas. En segundo lugar, añade trazabilidad y persistencia, permitiendo saber qué usuario realizó cada acción y consultar resultados históricos. Por último, incorpora una capa de correlación y scoring que ayuda a priorizar los hallazgos sin convertir la plataforma en una herramienta de explotación automática.

De este modo, la solución propuesta no compite directamente con herramientas aisladas ni con plataformas comerciales completas, sino que ofrece una alternativa intermedia, controlada y extensible, especialmente adecuada para entornos de laboratorio, formación avanzada y auditorías autorizadas de alcance limitado.

1.5 Valor añadido del proyecto

El principal valor añadido de la propuesta reside en su enfoque estructurado de la fase de pre-explotación y en su diseño orientado a evolucionar hacia un producto real. La integración de mecanismos de autenticación, auditoría y persistencia de resultados permite que la plataforma sea adecuada para entornos profesionales donde la trazabilidad y la reproducibilidad son requisitos fundamentales.

Además, el proyecto sienta una base sólida para futuras ampliaciones, como la incorporación de nuevos módulos de análisis, mejoras en los mecanismos de contención o la integración con proveedores de identidad externos, manteniendo siempre un enfoque ético y controlado.

2. INTRODUCCIÓN

Este proyecto aborda el desarrollo de una plataforma orientada a mejorar la fase de pre-explotación dentro de una auditoría de seguridad ofensiva. El objetivo principal es proporcionar una visión clara y estructurada sobre la superficie de ataque de un binario ejecutable antes de entrar en fases posteriores del análisis, facilitando la identificación de información relevante desde un punto de vista técnico.

En muchos entornos, esta fase se realiza mediante el uso de herramientas independientes y análisis manual, lo que dificulta la correlación de resultados y reduce la trazabilidad del trabajo realizado. La solución propuesta pretende centralizar este proceso, integrando análisis estático y análisis dinámico controlado, y almacenando los resultados de forma persistente para su posterior consulta mediante una interfaz web.

Asimismo, al tratarse de un sistema diseñado con una visión de producto, la plataforma incorpora mecanismos de control de acceso y auditoría, permitiendo registrar quién realiza cada acción y cuándo. Este enfoque no solo mejora la organización del análisis, sino que también facilita la evolución futura del sistema hacia escenarios de uso más complejos.

2.1 Principales requisitos del proyecto:

- Control de acceso mediante usuarios autenticados y roles.
- Subida y gestión de binarios para su análisis.
- Análisis estático de binarios ELF y evaluación de mitigaciones de seguridad.
- Análisis dinámico controlado con registro de eventos relevantes.
- Almacenamiento persistente de resultados y consulta posterior.
- Generación de informes descargables.

- Registro de auditoría de acciones realizadas en el sistema.

3. OBJETIVOS

3.1 R01 – El programa debe permitir acceder solo a usuarios autorizados

R01F01 – El usuario debe poder registrarse en el sistema

- R01F01T01 – Crear la tabla users en la base de datos (id, email, password_hash, enabled, created_at).
 - R01F01T01P01 – Insertar usuario de prueba y verificar persistencia.
- R01F01T02 – Implementar servicio de registro con validaciones (email único, formato, password mínimo).
 - R01F01T02P01 – Probar registro con email duplicado y password débil (debe fallar).
- R01F01T03 – Implementar endpoint REST /auth/register.
 - R01F01T03P01 – Petición HTTP válida devuelve 201 y usuario creado.
- R01F01T04 – Crear pantalla de registro en el frontend con validación de formularios.
 - R01F01T04P01 – Visualizar formulario y validar campos requeridos.

R01F02 – El usuario debe iniciar sesión

- R01F02T01 – Implementar verificación de credenciales usando hash seguro (BCrypt/Argon2).
 - R01F02T01P01 – Login correcto/incorrecto con usuario real.
- R01F02T02 – Implementar emisión de token (JWT) con expiración.
 - R01F02T02P01 – Probar acceso a endpoint protegido con y sin token.
- R01F02T03 – Implementar pantalla de login y almacenamiento seguro del token en frontend.
 - R01F02T03P01 – Login desde UI y navegación a zona privada.

R01F03 – El sistema debe controlar permisos por roles

- R01F03T01 – Crear tablas roles y user_roles y roles base (ADMIN/ANALYST/VIEWER).
 - R01F03T01P01 – Asignar rol a usuario y verificar en DB.
- R01F03T02 – Proteger endpoints por rol (Spring Security).
 - R01F03T02P01 – Un VIEWER no puede lanzar análisis (403).

R01F04 – (Opcional) El sistema debe poder integrarse con Auth0

- R01F04T01 – Documentar integración OIDC (redirect URIs, scopes, issuer, audience).
 - R01F04T01P01 – Checklist de configuración completado.

3.2 R02 – El programa debe poder guardar datos y mantener historiales de análisis

R02F01 – El sistema debe disponer de base de datos implantada

- R02F01T01 – Preparar PostgreSQL (docker dev) y parámetros de conexión.
 - R02F01T01P01 – Arrancar DB y conectar desde backend.
- R02F01T02 – Configurar migraciones (Flyway/Liquibase).
 - R02F01T02P01 – Ejecutar migraciones en entorno limpio sin errores.

R02F02 – El sistema debe guardar análisis y resultados

- R02F02T01 – Crear tablas analyses (metadatos) y analysis_results (JSON/JSONB).
 - R02F02T01P01 – Crear un análisis y guardar un results de prueba.
- R02F02T02 – Crear índices mínimos (analysis_id, created_at, hash).
 - R02F02T02P01 – Consultar listados por fecha sin degradación evidente.

3.3 R03 – El programa debe permitir subir binarios para su análisis

R03F01 – El usuario debe poder subir un ejecutable

- R03F01T01 – Implementar endpoint /binaries/upload (multipart) con límite de tamaño.
 - R03F01T01P01 – Subir binario válido y uno demasiado grande (debe rechazar).

- R03F01T02 – Almacenar archivo con ruta segura y nombre interno (no el original).
- R03F01T02P01 – Verificar que no hay path traversal y que el archivo existe.
- R03F01T03 – Calcular SHA-256 y asociarlo al analysis.
- R03F01T03P01 – Hash coincide con herramienta externa.

R03F02 – El sistema debe identificar el tipo de binario

- R03F02T01 – Detectar formato por magic bytes (ELF/PE/unknown).
- R03F02T01P01 – Probar con fixtures ELF, PE y fichero aleatorio.

3.4 R04 – El programa debe analizar el binario de forma estática (ELF MVP)

R04F01 – El sistema debe extraer información estructural

- R04F01T01 – Parsear cabecera ELF (arquitectura, entrypt, tipo).
- R04F01T01P01 – Comparar con readelf -h en binario de prueba.
- R04F01T02 – Listar secciones/segmentos con tamaños.
- R04F01T02P01 – Comparar con readelf -S en muestra.

R04F02 – El sistema debe evaluar mitigaciones

- R04F02T01 – Determinar NX (segmentos/flags ejecutables).
- R04F02T01P01 – Validar contra herramienta de referencia (p.ej. checksec).
- R04F02T02 – Determinar PIE (tipo de ejecutable).
- R04F02T02P01 – Validar con binarios PIE/no PIE conocidos.
- R04F02T03 – Determinar RELRO (parcial/completo).
- R04F02T03P01 – Validar contra checksec.

R04F03 – El sistema debe extraer strings relevantes

- R04F03T01 – Extraer strings con filtros (minlen, charset) y resumen.
- R04F03T01P01 – Verificar conteo y ejemplos esperados.
- R04F03T02 – Marcar “strings interesantes” (URLs, paths, tokens).
- R04F03T02P01 – Fixture con patrones → deben detectarse.

3.5 R05 – El programa debe ejecutar un análisis dinámico controlado

R05F01 – El sistema debe ejecutar con control de tiempo

- R05F01T01 – Ejecutar el binario con timeout y finalización segura.
- R05F01T01P01 – Binario “sleep infinito” debe ser terminado.

R05F02 – El sistema debe registrar llamadas al sistema

- R05F02T01 – Implementar agente C (ptrace) que capture syscalls básicas.
- R05F02T01P01 – Binario de prueba genera syscalls y se registran.
- R05F02T02 – Emitir eventos en formato estructurado (JSON lines).
- R05F02T02P01 – Validar que el output es JSON válido y parseable.

R05F03 – El sistema debe registrar accesos a ficheros

- R05F03T01 – Detectar syscalls relacionadas con FS y registrar paths cuando sea posible.
- R05F03T01P01 – Binario que abre un fichero debe reflejar evento.

3.6 R06 – El programa debe guardar y mostrar resultados del análisis

R06F01 – El sistema debe almacenar resultados

- R06F01T01 – Persistir results (estático+dinámico) en analysis_results como JSON/JSONB.
- R06F01T01P01 – Guardar y recuperar sin pérdida/alteración.

R06F02 – El usuario debe visualizar resultados

- R06F02T01 – Implementar endpoint /analyses/{id} con resultados.
- R06F02T01P01 – Respuesta contiene secciones estático/dinámico.
- R06F02T02 – Crear vista UI con pestañas (Resumen/Estático/Dinámico).
- R06F02T02P01 – Navegación por tabs correcta.

3.7 R07 – El programa debe generar un informe del análisis

R07F01 – El sistema debe generar informe HTML

- R07F01T01 – Generar HTML con secciones y hallazgos principales.
- R07F01T01P01 – Abrir HTML y verificar contenido mínimo.

- R07F01T02 – Permitir descarga del informe.
- R07F01T02P01 – Descargar y verificar integridad.

3.8 R08 – El programa debe registrar quién realiza cada acción (auditoría)

R08F01 – El sistema debe registrar eventos de auditoría

- R08F01T01 – Crear tabla audit_events (user_id, action, timestamp, analysis_id, result).
- R08F01T01P01 – Insertar evento manual y consultarlo.
- R08F01T02 – Registrar automáticamente eventos en acciones clave (login, upload, analysis start).
- R08F01T02P01 – Realizar flujo y comprobar eventos generados.
- R08F01T03 – Endpoint de consulta para ADMIN con filtros/paginación.
- R08F01T03P01 – Consultar por fecha/usuario y validar paginado.

4. DESCRIPCIÓN

4.1 Arquitectura de la solución.

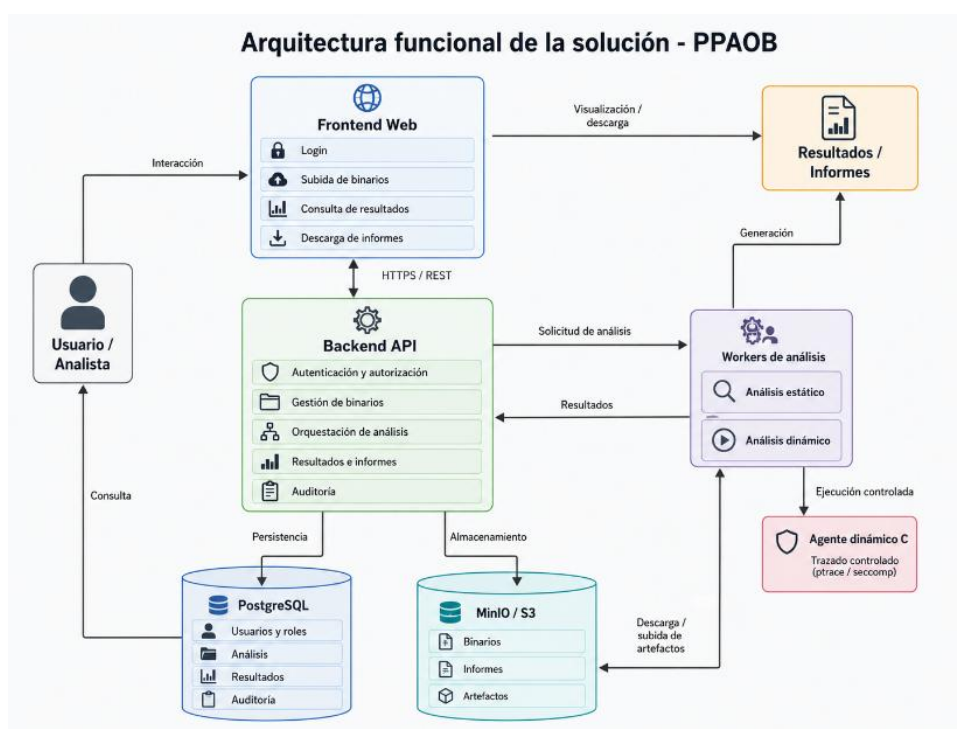
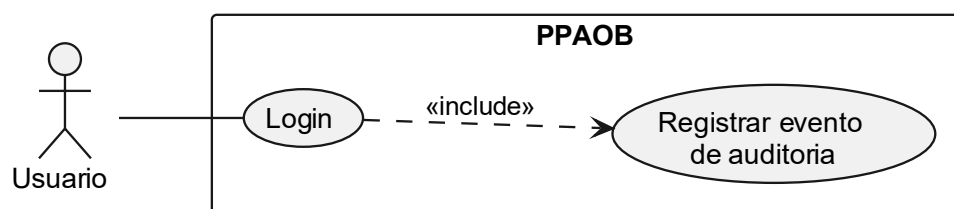


Ilustración 1: Diagrama Arquitectura de la solución (Fuente: elaboración propia)

4.2 Casos de uso de la aplicación

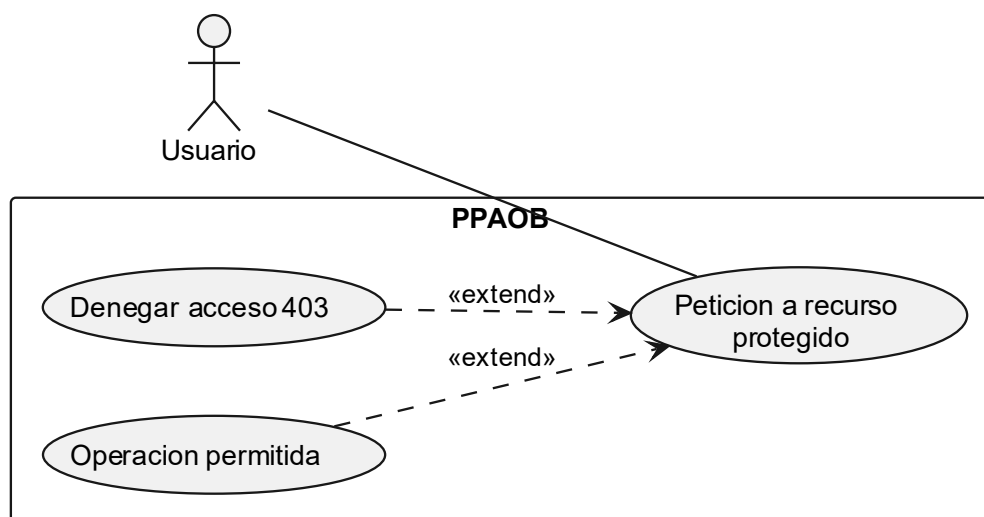
CU00 Autenticar Usuario:



DESCRIPCIÓN: 1. El usuario abre `#/login`. 2. Introduce credenciales. 3. El sistema valida email/password. 4. El sistema emite JWT y refresh token. 5. El sistema devuelve perfil básico.poli	
PRECONDICIONES: Usuario registrado y habilitado	POSTCONDICIONES: Access token emitido, cookie refresh rotada, auditoria de login
DATOS ENTRADA Email, contraseña	DATOS SALIDA accessToken, user, cookie 'refresh_token'
TABLAS: USERS, REFRESH_SESSIONS, AUDIT_EVENTS	CLASES: AuthController, AuthService, JwtService, JdbcRefreshSessionRepository
INTERFACES: POST /api/v1/auth/login GET /api/v1/auth/me	

Tabla 2: caso de uso Autenticar Usuario

CU00-A Autorizar Operación por Rol:



DESCRIPCIÓN:

1. El actor llama a un endpoint protegido.
2. El filtro JWT valida token y extrae claims.
3. Security compara roles con política del endpoint.
4. Si cumple, deja ejecutar; si no, bloquea con 403.

PRECONDICIONES:

JWT valido y endpoint protegido

POSTCONDICIONES:

Operación HTTP permitida o denegada.

DATOS ENTRADA

JWT, recurso solicitado, política de acceso

DATOS SALIDA

ALLOW o DENY

TABLAS:

USERS, USER_ROLES, ROLES

CLASES:

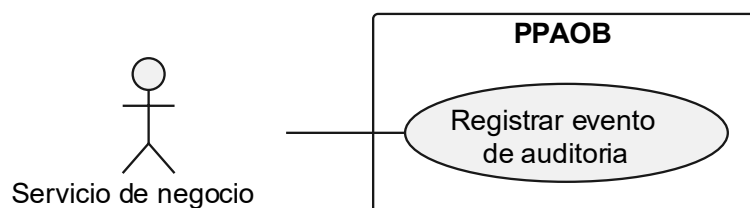
SecurityConfig, JwtAuthenticationFilter, JwtService

INTERFACES:

Aplicado a endpoints ANALYST/ ADMIN y ADMIN.

Tabla 3: caso de uso Autorizar Operación por Rol

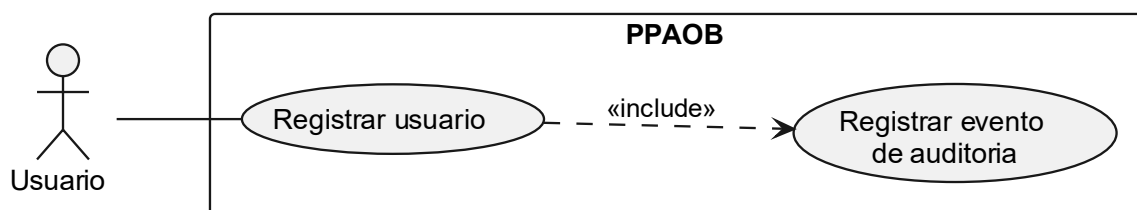
CU00-B Registrar Evento Auditoría:



DESCRIPCIÓN: 1. Se ejecuta una accion clave (auth/upload/analysis/report/admin). 2. El servicio de negocio construye evento auditado. 3. El repositorio persiste evento con contexto.	
PRECONDICIONES: Acción de negocio ejecutada	POSTCONDICIONES: Evento append-only persistido
DATOS ENTRADA ‘action’, ‘result’, ‘userId’ y contexto	DATOS SALIDA ‘audit_event_id’ persistido
TABLAS: AUDIT_EVENTS	CLASES: AuditService, JdbcAuditEventRepository
INTERFACES: GET /api/v1/audit/events (consulta posterior)	

Tabla 4: caso de uso Registrar evento de auditoría

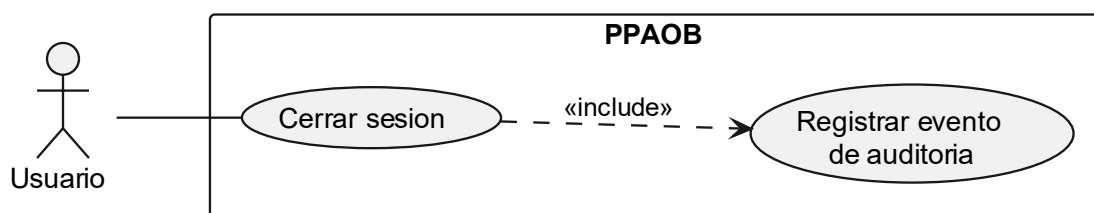
CU01 Registrar Usuario:



DESCRIPCIÓN:	
1. Usuario abre `#/register`. 2. Introduce email y password. 3. Sistema valida formato y unicidad. 4. Sistema hashlea password y persiste usuario.	
PRECONDICIONES:	POSTCONDICIONES:
Email no registrado	Usuario creado
DATOS ENTRADA	DATOS SALIDA
Email, contraseña	userId, email, createdAt
TABLAS:	CLASES:
USERS	AuthController, AuthService, JdbcUserAccountRepository
INTERFACES:	
POST /api/v1/auth/register	

Tabla 5: caso de uso Registrar Usuario

CU02 Cerrar Sesión:



DESCRIPCIÓN:

1. Usuario invoca logout.
2. Sistema revoca refresh session vigente.
3. Sistema limpia cookie HttpOnly.
4. Se registra evento de auditoria.

PRECONDICIONES:

Sesión refresh activa

POSTCONDICIONES:

Refresh revocado y cookie limpia

DATOS ENTRADA

Cookie 'refresh_token'

DATOS SALIDA

204 sin contenido

TABLAS:

REFRESH_SESSIONS, AUDIT_EVENTS

CLASES:

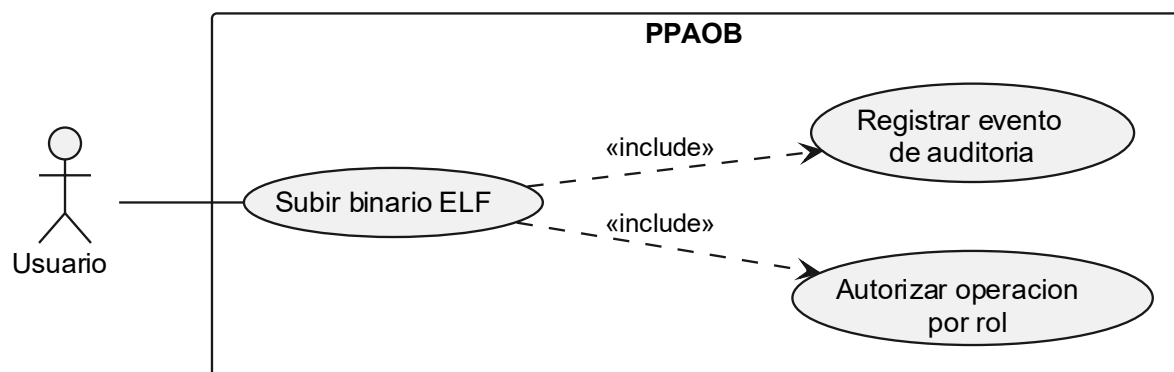
AuthController, AuthService

INTERFACES:

POST /api/v1/auth/logout

Tabla 6: caso de uso Cerrar Sesión

CU03 Subir Binario:



DESCRIPCIÓN:

1. Usuario envía multipart a binaries.
2. Sistema valida tamaño y formato ELF.
3. Calcula SHA-256 y aplica deduplicación.
4. Guarda objeto en S3/MinIO (si nuevo).
5. Persiste metadatos y enlace de ownership.
6. Registra auditoria.

PRECONDICIONES:

Usuario autenticado

POSTCONDICIONES:

Binario almacenado/reutilizado y vinculado al usuario

DATOS ENTRADA

File (multipart)

DATOS SALIDA

BinaryId, sha256, restoredObject

TABLAS:

STORED_OBJECTS, BINARIES,
BINARY_UPLOADS, AUDIT_EVENTS

CLASES:

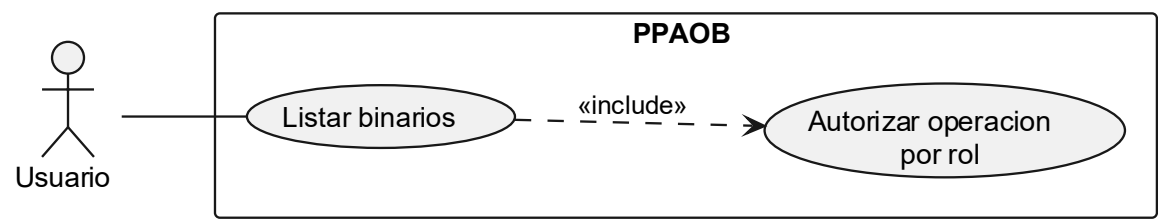
BinaryController, BinaryService,
S3ObjectStorageAdapter

INTERFACES:

POST /api/v1/binaries

Tabla 7: caso de uso Subir binario ELF

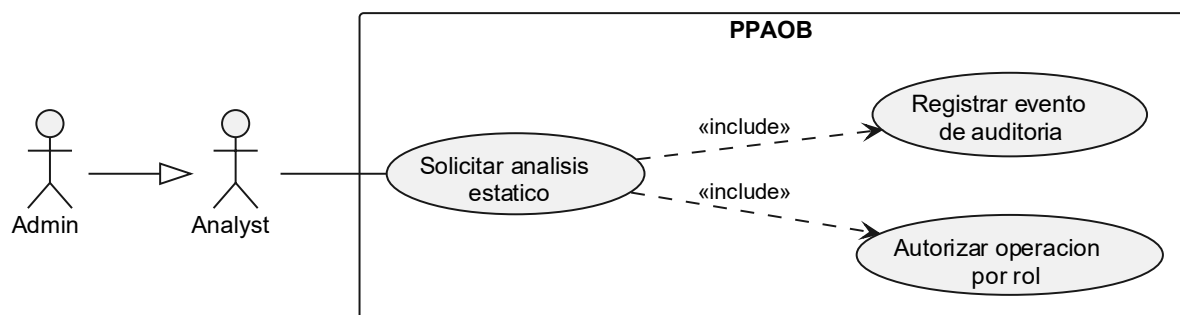
CU04 Listar Binarios:



DESCRIPCIÓN: 1. Usuario solicita listado de binarios. 2. Sistema filtra por ownership del actor. 3. Devuelve metadata y disponibilidad de objeto.	
PRECONDICIONES: Usuario autenticado	POSTCONDICIONES: Listado devuelto
DATOS ENTRADA Token de sesión	DATOS SALIDA Lista de binarios de usuario
TABLAS: BINARIES, BINARY_UPLOADS, STORED_OBJECTS	CLASES: BinaryController, BinaryService
INTERFACES: GET /api/v1/binaries	

Tabla 8: caso de uso Listar binarios de usuario

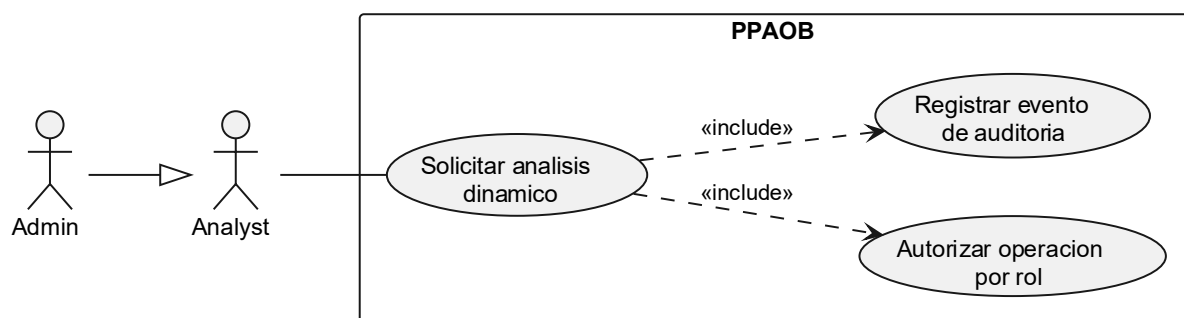
CU05 Solicitar Análisis Estático:



DESCRIPCIÓN: 1. Actor solicita analisis con `profile=STATIC_BASELINE`. 2. Sistema valida permisos y ownership. 3. Crea job en `analyses`. 4. Worker static reclama y ejecuta. 5. Persiste `analysis_results` y marca `DONE/FAILED`.	
PRECONDICIONES: Binario disponible en storage	POSTCONDICIONES: Análisis `PENDING/RUNNING` creado y auditable
DATOS ENTRADA binaryId, profile= STATIC_BASELINE	DATOS SALIDA analysisId, status
TABLAS: ANALYSES, ANALYSIS_RESULTS, AUDIT_EVENTS	CLASES: AnalysisController, AnalysisService, worker static services
INTERFACES: POST /api/v1/analyses, GET /api/v1/analyses/{id}	

Tabla 9: Solicitar análisis estático

CU06 Solicitar Análisis Dinámico:



DESCRIPCIÓN:

1. Actor solicita analisis con `profile=DYNAMIC\_BASELINE`.
2. Worker dinamico reclama job y ejecuta agente C con timeout.
3. Se generan eventos syscall NDJSON + resumen runtime/policy/fs.
4. Se almacena traza completa como `DYNAMIC\_TRACE`.
5. Se persiste resumen en `analysis\_results`.

PRECONDICIONES:

Binario disponible en storage y entorno dinámico habilitado

POSTCONDICIONES:

Resultado dinámico y artifact de traza persistidos

DATOS ENTRADA

binaryId, profile= DYNAMIC_BASELINE

DATOS SALIDA

analysisId, status, referencias de artifacts

TABLAS:

ANALYSES, ANALYSIS_RESULTS, STORED_OBJECTS, ARTIFACTS

CLASES:

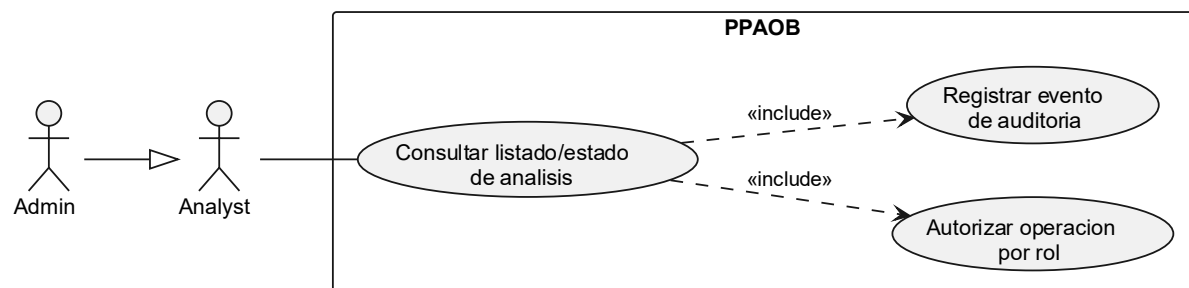
AnalysisController, AnalysisService, worker dynamic services, agent-dynamic-c

INTERFACES:

POST /api/v1/analyses, GET /api/v1/analyses/{id}

Tabla 10: caso de uso Solicitar análisis dinámico

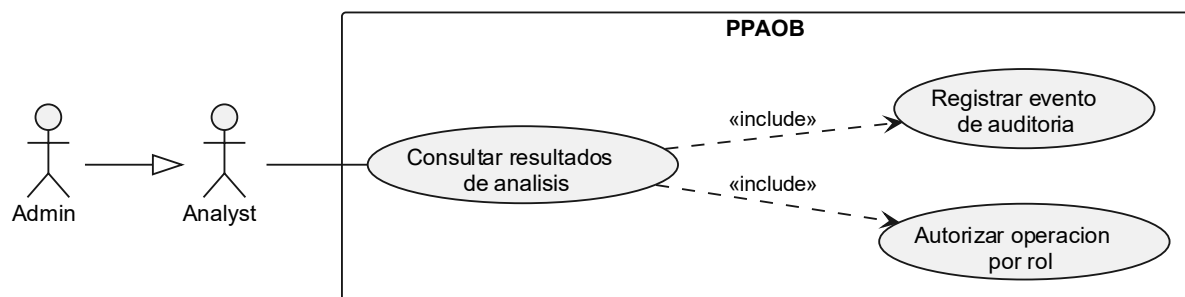
CU07 Consultar Análisis:



DESCRIPCIÓN: 1. Actor consulta listado de analisis. 2. Sistema aplica filtros opcionales. 3. Actor abre detalle de un analisis. 4. Sistema devuelve status y metadatos.	
PRECONDICIONES: Analisis existente y usuario autorizado	POSTCONDICIONES: Estado/lista visibles para triage
DATOS ENTRADA Filtros(status, binaryId)	DATOS SALIDA Lista/Detalle del análists
TABLAS: ANALYSES	CLASES: AnalysisController, AnalysisService, JdbcAnalysisRepository
INTERFACES: GET /api/v1/analyses, GET /api/v1/analyses/{id}	

Tabla 11: caso de uso Consultar análisis

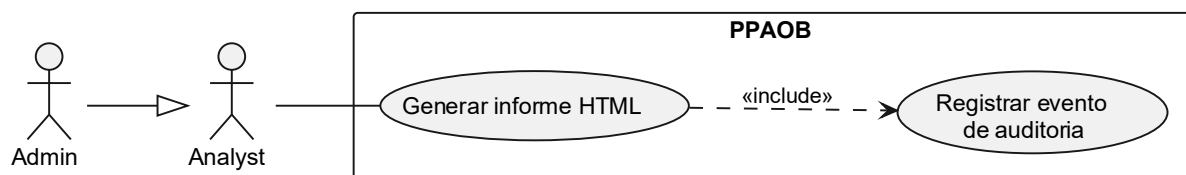
CU08 Consultar Resultados:



DESCRIPCIÓN: 1. Actor abre resultados de analisis. 2. Sistema recupera `analysis_results`. 3. Frontend muestra tabs Summary/Static/Dynamic/Signals/Correlation/Artifacts/Raw.	
PRECONDICIONES: Análisis en estado DONE	POSTCONDICIONES: Results_json entregado para visualización
DATOS ENTRADA analysisId	DATOS SALIDA Results_json
TABLAS: ANALYSIS_RESULTS	CLASES: AnalysisController, AnalysisService, AnalysesPage
INTERFACES: GET /api/v1/analyses/{analysesId}/results	

Tabla 12: caso de Uso Consultar resultados de análisis

CU09 Generar Informe HTML:



DESCRIPCIÓN:

1. Actor solicita generar reporte de un analisis.
2. Sistema renderiza HTML desde `results\_json`.
3. Almacena objeto en S3/MinIO.
4. Persiste metadata en `artifacts`.

PRECONDICIONES:

Análisis DONE con resultados persistidos

POSTCONDICIONES:

Artifact HTML generado y almacenado

DATOS ENTRADA

analysisId

DATOS SALIDA

artifactId, checksumSha256

TABLAS:

ANALYSIS_RESULTS,
 STORED_OBJECTS, ARTIFACTS

CLASES:

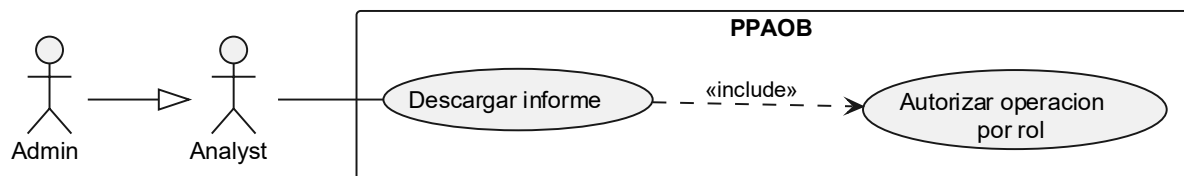
ReportController, ReportService,
 S3ObjectStorageAdapter

INTERFACES:

POST /api/v1/reports

Tabla 13: caso de uso Generar Informe HTML

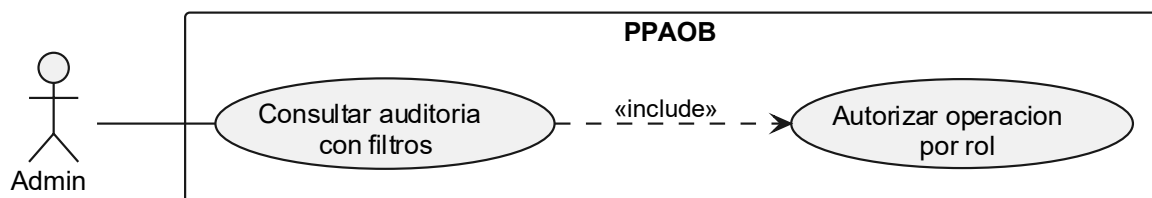
CU10 Descargar Informe:



DESCRIPCIÓN: 1. Actor solicita descarga por `artifactId`. 2. Sistema valida ownership/rol. 3. Descarga bytes desde object storage y responde al cliente.	
PRECONDICIONES: Artifact de tipo REPORT_HTML existente y autorizado	POSTCONDICIONES: Archivo descargado por el actor
DATOS ENTRADA artifactId	DATOS SALIDA Text/html descargable
TABLAS: ARTIFACTS, STORED_OBJECTS	CLASES: ReportController, ReportService, S3ObjectStorageAdapter
INTERFACES: GET /api/v1/reports/{artifactId}/download	

Tabla 14: caso de uso Descargar Informe HTML

CU11 Consultar Auditoría:



DESCRIPCIÓN:

1. Admin abre `#/audit`.
2. Define filtros (`action`, `result`, `userId`, `analysisId`, `binaryId`, `from`, `to`).
3. Sistema aplica paginacion `limit/offset`.
4. Devuelve eventos ordenados.

PRECONDICIONES:

Actor con rol ADMIN

POSTCONDICIONES:

Eventos auditados filtrados/paginados

DATOS ENTRADA

Parámetros de filtro y paginación

DATOS SALIDA

Lista de eventos de auditoría

TABLAS:

AUDIT_EVENTS

CLASES:

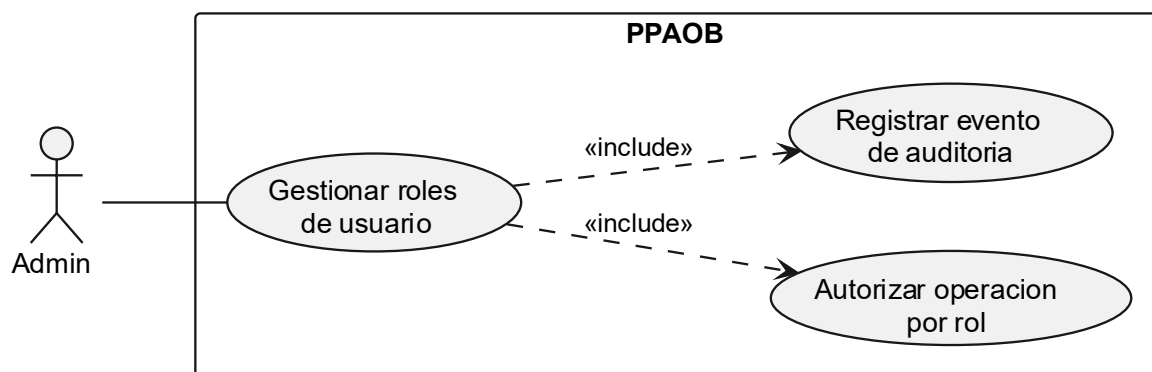
AuditController, AuditService,
JdbcAuditEventRepository

INTERFACES:

GET /api/v1/audit/events

Tabla 15: caso de uso Consultar auditoría con filtros

CU12 Gestionar Roles Usuarios:



DESCRIPCIÓN:

1. Admin abre `#/admin-users` y selecciona usuario.
2. Modifica conjunto de roles.
3. Sistema valida reglas de seguridad.
4. Persiste `user\_roles` y registra `USER\_ROLE\_UPDATE`.

PRECONDICIONES:

Actor con rol ADMIN

POSTCONDICIONES:

Roles del usuario objetivo actualizados y auditados

DATOS ENTRADA

userId, roles[]

DATOS SALIDA

Usuario actualizado

TABLAS:

AUDIT_EVENTS

CLASES:

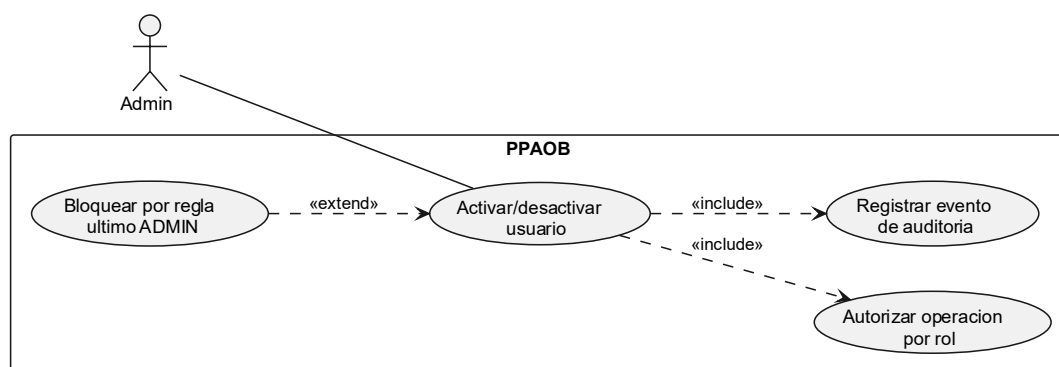
AdminController, AdminUserService, JdbcUserAccountRepository

INTERFACES:

PUT /api/v1/admin/users/{userId}/roles

Tabla 16: caso de uso Gestionar roles de usuario

CU13 Gestionar Usuarios Habilitados:



DESCRIPCIÓN: 1. Admin selecciona usuario en `#/admin-users`. 2. Envía cambio de `enabled`. 3. Sistema comprueba regla de último ADMIN habilitado. 4. Si cumple, persiste cambio y audita `USER_ENABLED_UPDATE`. 5. Si no cumple, rechaza operación.	
PRECONDICIONES: Usuario con rol ADMIN	POSTCONDICIONES: Estado enabled actualizado o bloqueo si check de último admin.
DATOS ENTRADA userId, enabled	DATOS SALIDA Usuario actualizado o error de regla
TABLAS: USERS, USER_ROLES, AUDIT_EVENTS	CLASES: AdminUserController, AdminUserService
INTERFACES: PATCH /api/v1/admin/{userId}/enabled	

Tabla 17: caso de uso Activar o desactivar usuario

Ilustración 3: Diagrama de clases Worker Static (Fuente: Elaboración propia)

5.3 Diagrama de clases (Worker Dynamic):

Ruta en el Proyecto: /docs/workers/dynamic/class_diagram.png

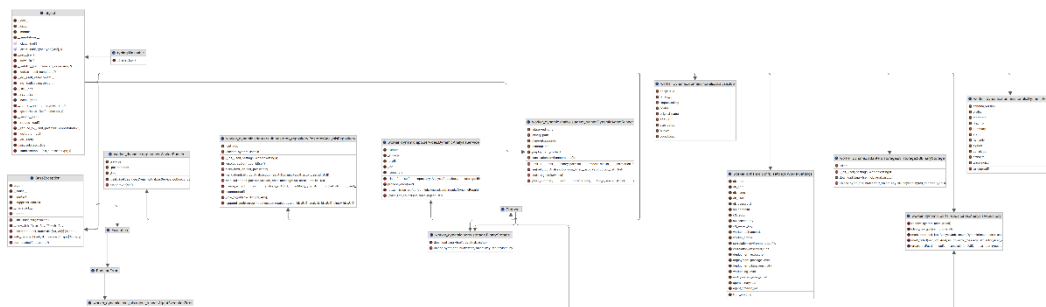


Ilustración 4: Diagrama de clases Worker Dynamic (Fuente: Elaboración propia)

5.4 Diagrama E/R (Entidad - Relación)

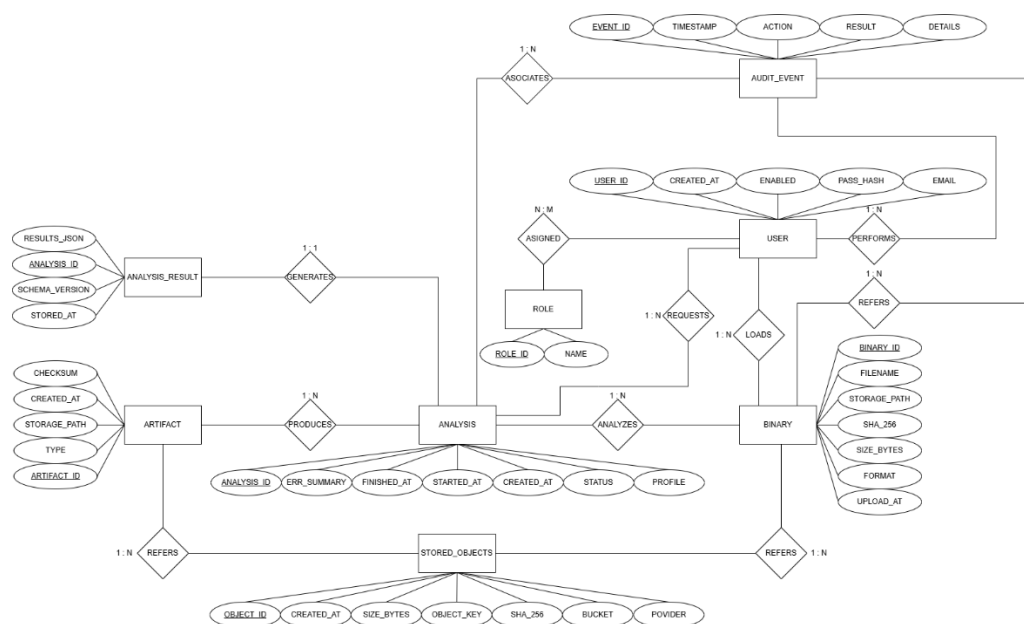


Ilustración 5: Diagrama de Entidad Relación (Fuente: Elaboración propia)

5.5 Diagrama de la base de datos.

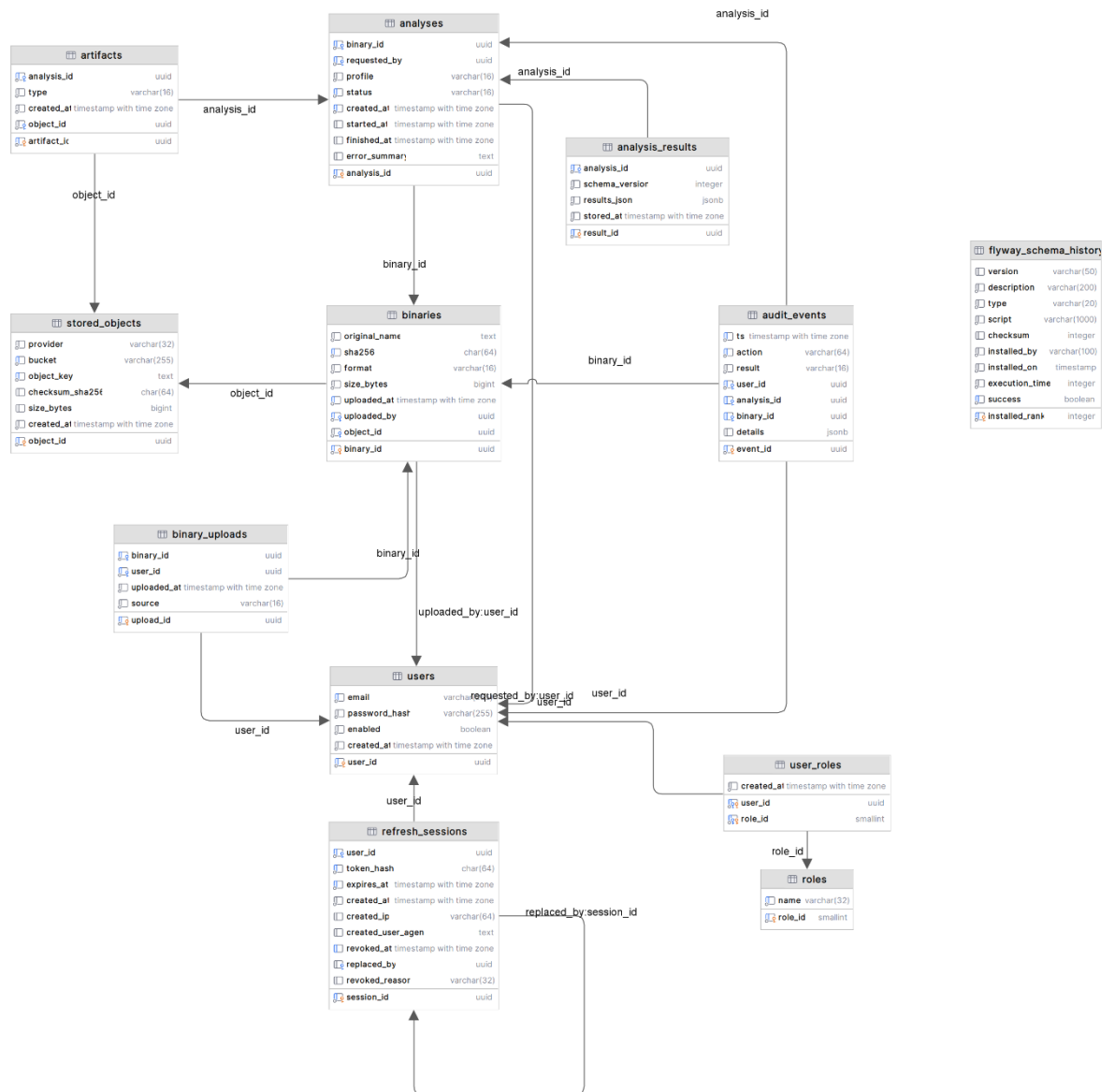


Ilustración 6: Diagrama de tablas de Bases de datos (Fuente: Elaboración propia)

5.6 Diagrama de flujo de navegación.

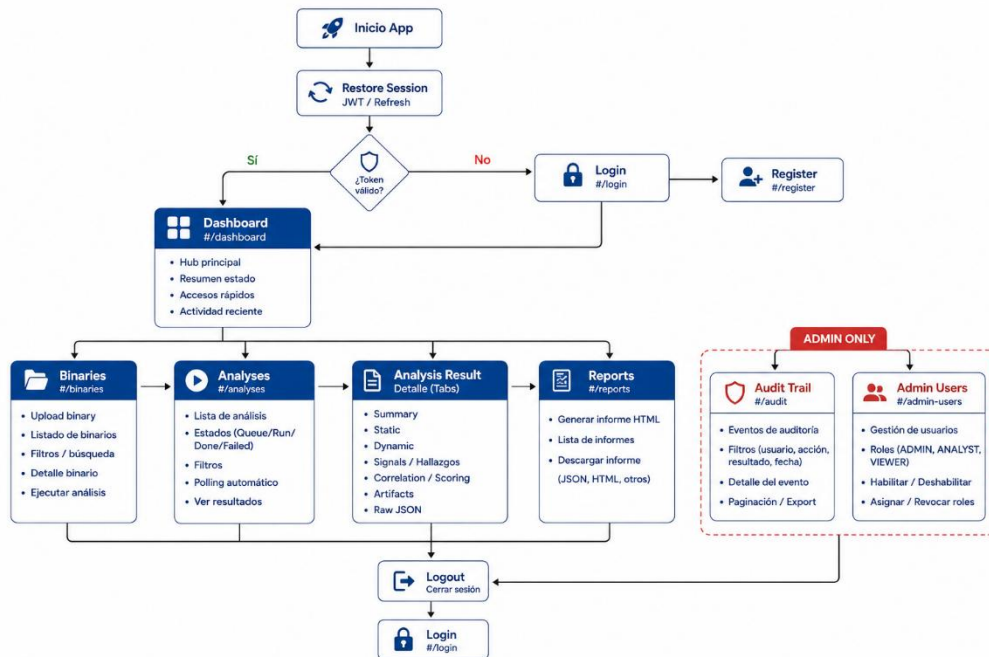
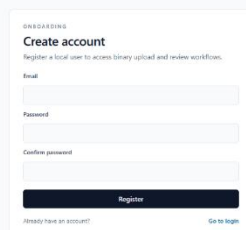


Ilustración 7: Diagrama de Flujo de Navegación (Fuente Elaboración propia)

5.7 Interfaces.



WELCOME

Create account

Register a local user to access library upload and review workflows.

Email

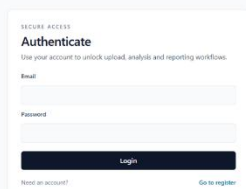
Password

Confirm password

Register

[Already have an account?](#) [Go to login](#)

Ilustración 8: Interfaz de Login Genérica (Fuente: Elaboración propia)



SECURE ACCESS

Authenticate

Use your account to unlock upload, analysis and reporting workflows.

Email

Password

Login

[Forgot an account?](#) [Go to register](#)

Ilustración 9: Interfaz de Registro Genérica (Fuente: Elaboración propia)

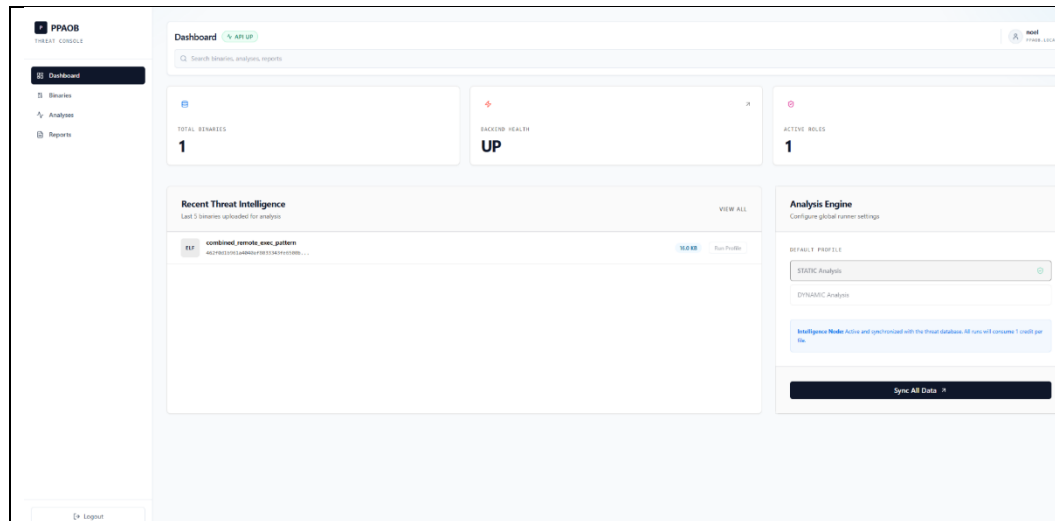


Ilustración 10: Interfaz Dashboard Rol Viewer (Fuente: Elaboración propia)

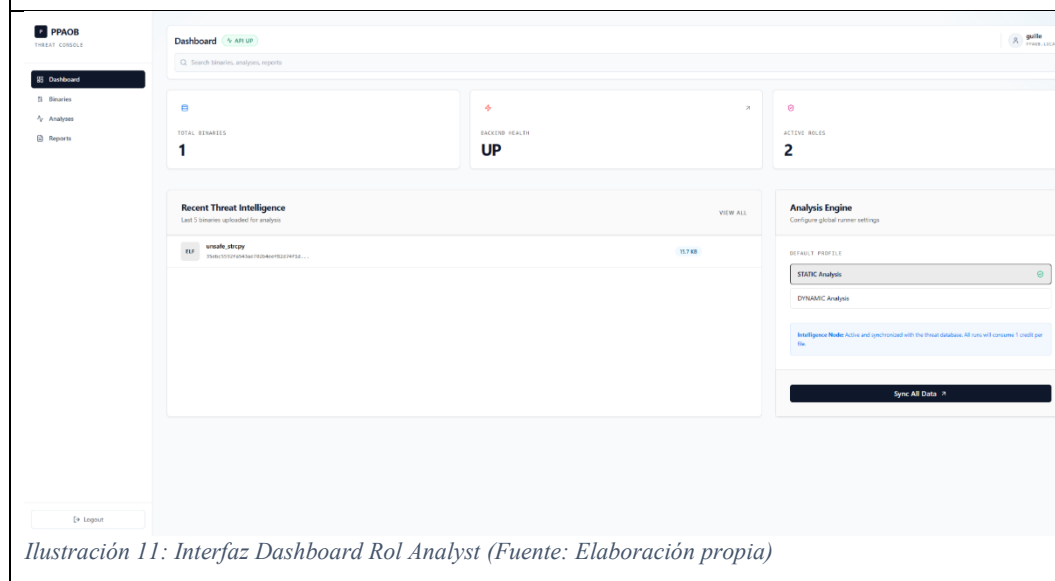


Ilustración 11: Interfaz Dashboard Rol Analyst (Fuente: Elaboración propia)

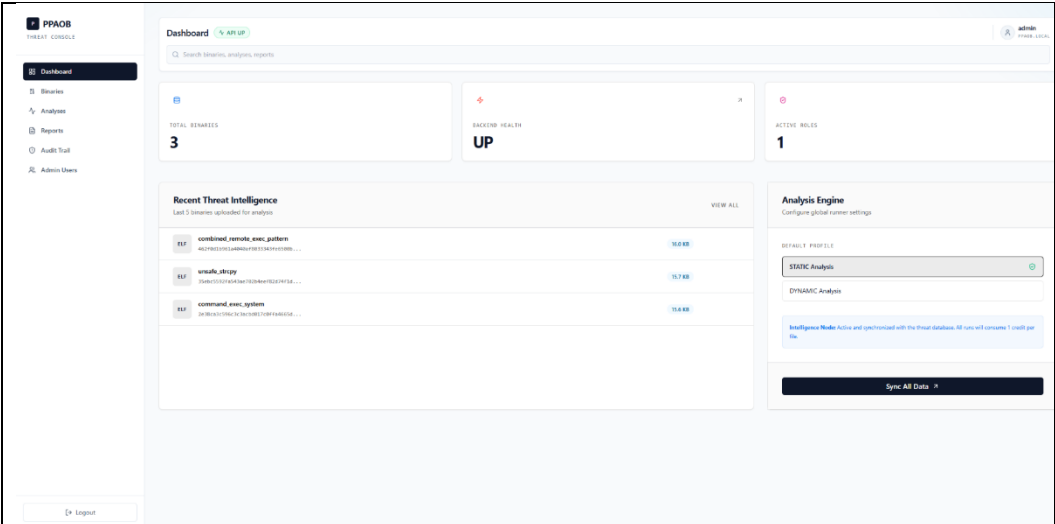


Ilustración 12: Interfaz Dashboard Rol Admin (Fuente: Elaboración propia)

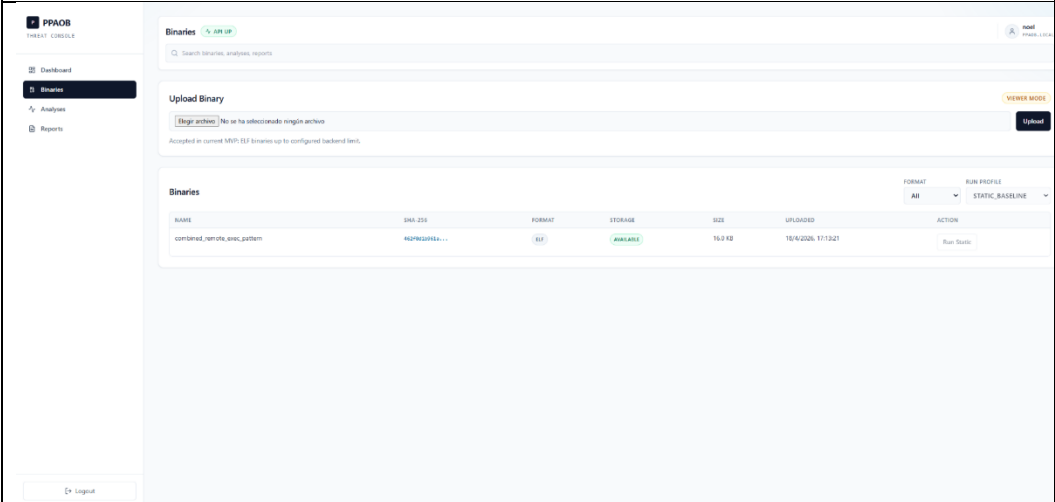


Ilustración 13: Interfaz Binaries Rol Viewer (Fuente: Elaboración propia)

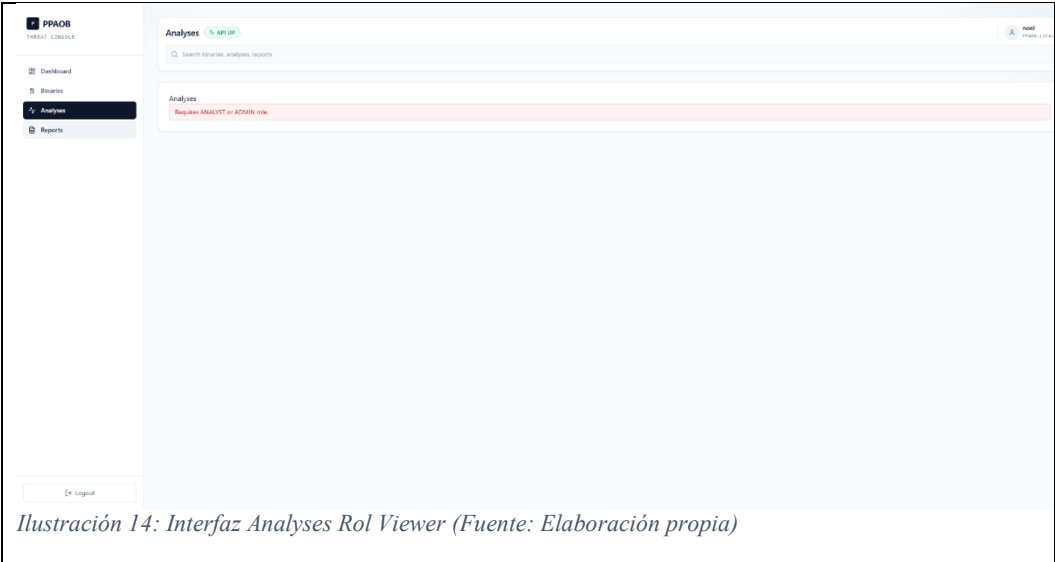


Ilustración 14: Interfaz Analyses Rol Viewer (Fuente: Elaboración propia)

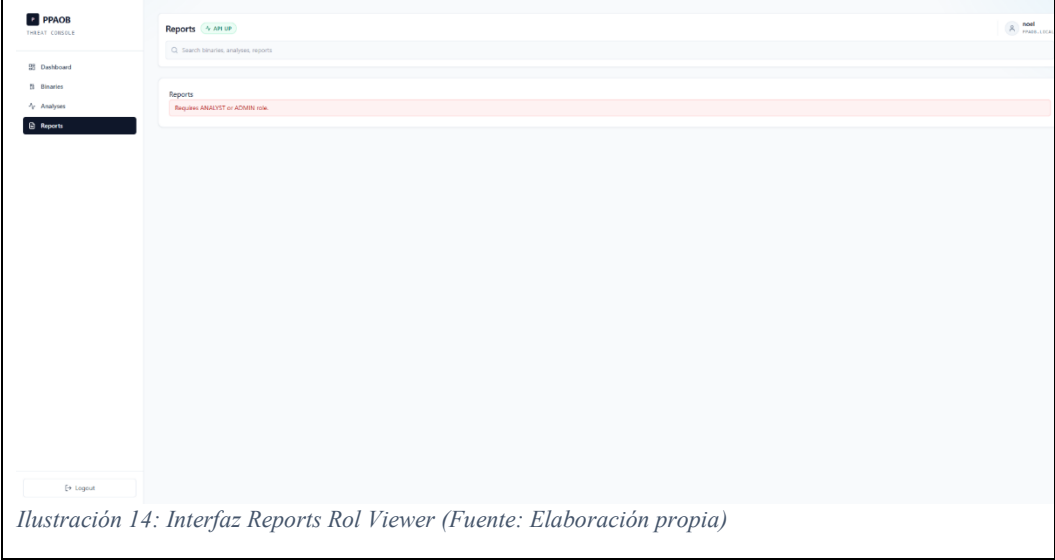


Ilustración 14: Interfaz Reports Rol Viewer (Fuente: Elaboración propia)

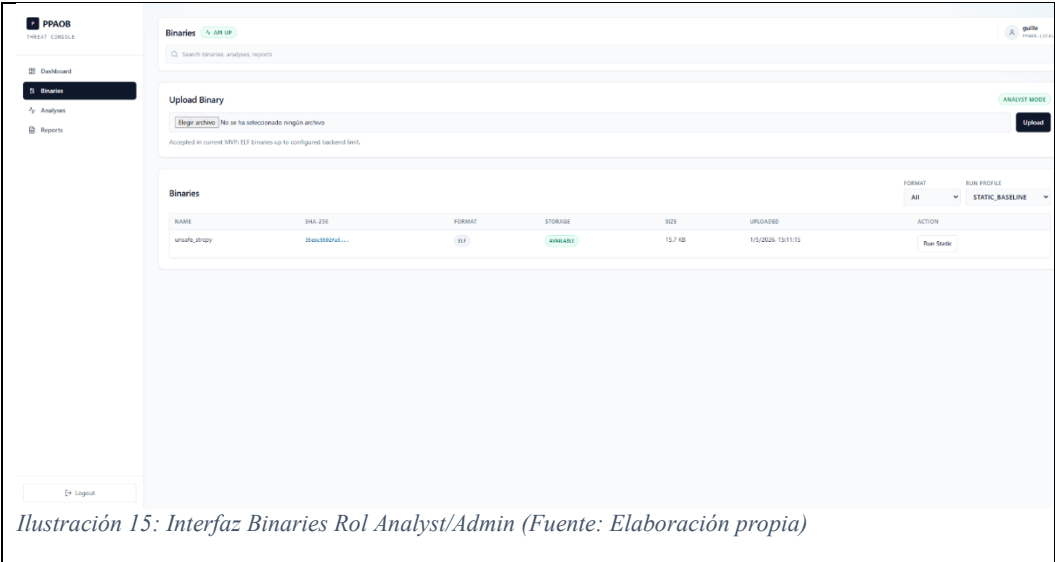


Ilustración 15: Interfaz Binaries Rol Analyst/Admin (Fuente: Elaboración propia)

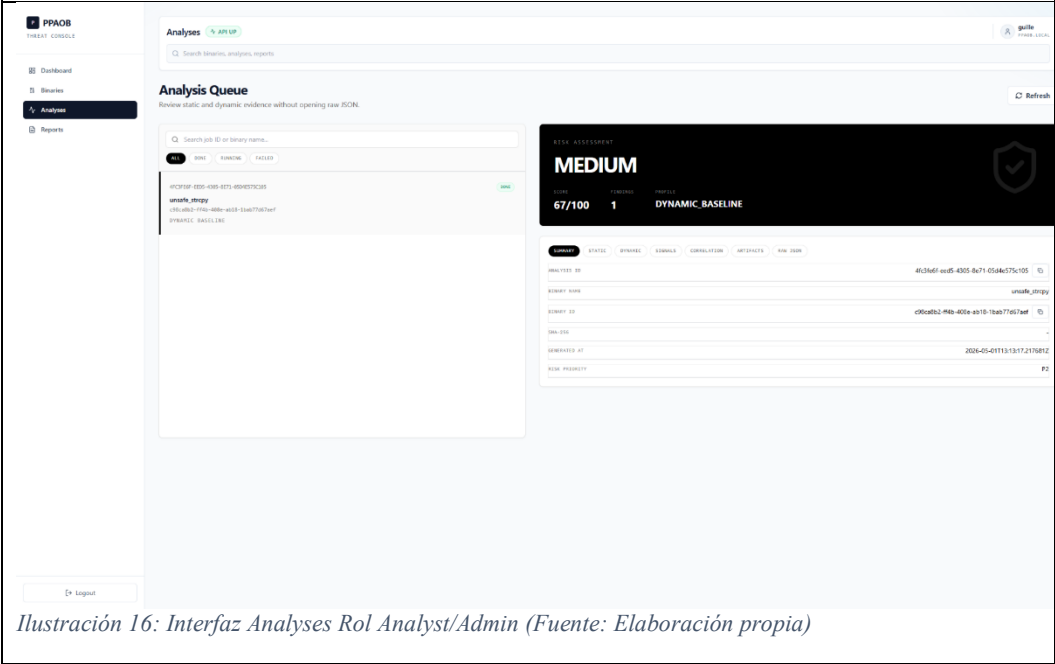


Ilustración 16: Interfaz Analyses Rol Analyst/Admin (Fuente: Elaboración propia)

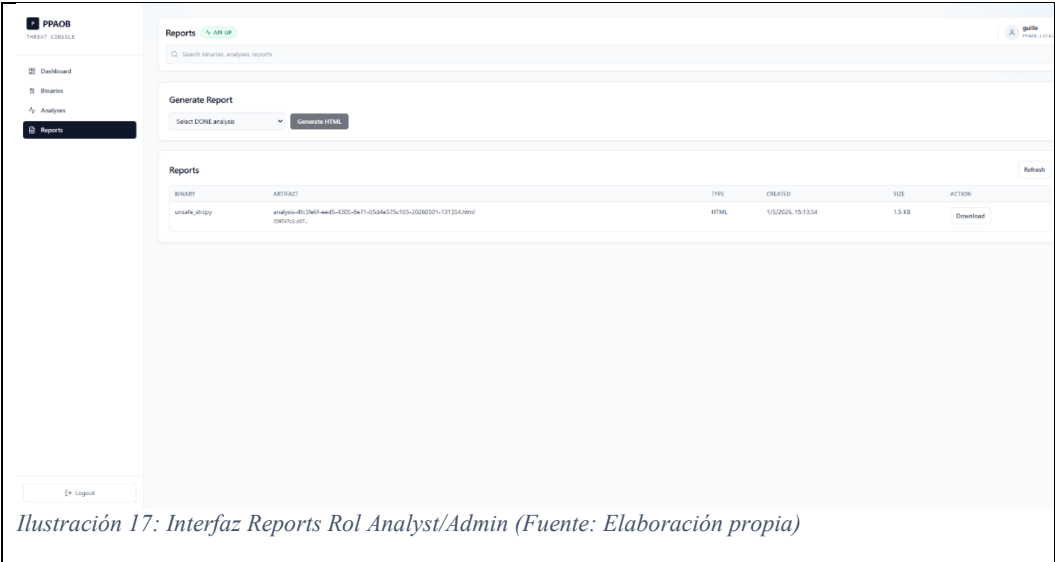


Ilustración 17: Interfaz Reports Rol Analyst/Admin (Fuente: Elaboración propia)

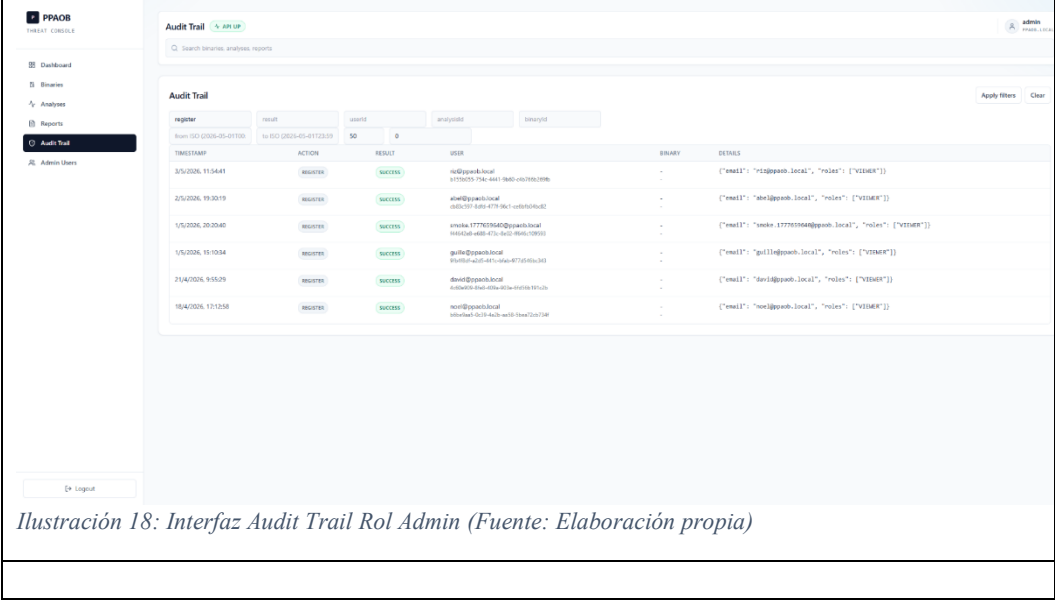


Ilustración 18: Interfaz Audit Trail Rol Admin (Fuente: Elaboración propia)

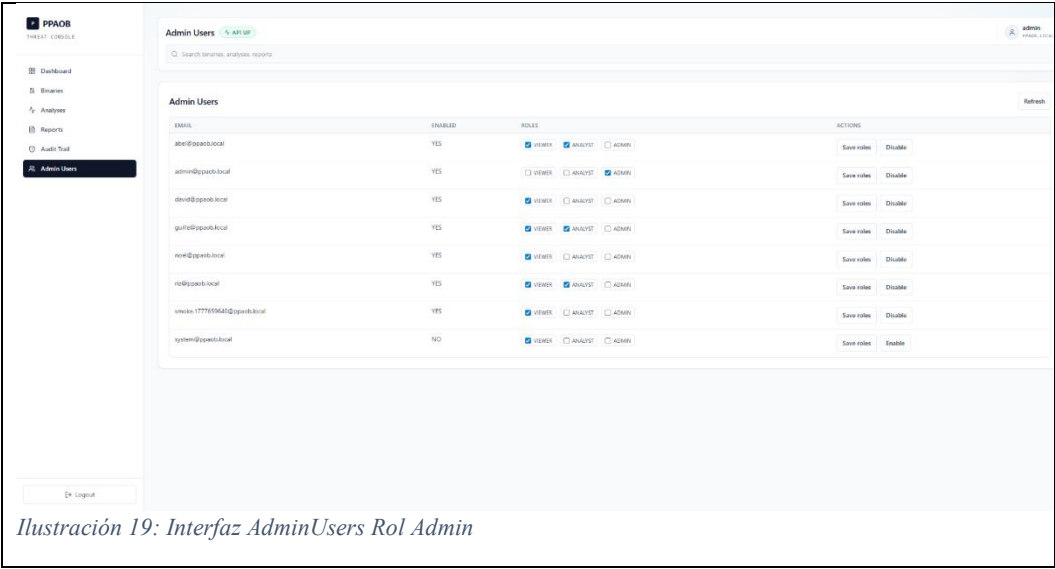


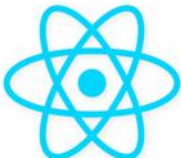
Ilustración 19: Interfaz AdminUsers Rol Admin

6. TECNOLOGÍA

Las tecnologías y herramientas utilizadas para este proyecto. Por ejemplo:

	Java Java es un lenguaje de programación orientado a objetos, robusto y ampliamente utilizado en aplicaciones empresariales. Se utiliza como lenguaje principal del backend. Sobre Java se implementan los servicios de autenticación, gestión de binarios, análisis, informes y auditoría. Aporta estabilidad, mantenibilidad y una base adecuada para organizar una API backend compleja.
	Spring Boot Spring Boot es un framework de Java que facilita la creación de aplicaciones backend y APIs REST. En el proyecto se utiliza para construir la API del backend, encargada de coordinar la lógica principal de la plataforma y exponer los endpoints consumidos por el frontend. Aporta rapidez de desarrollo, estructura y facilidad de configuración.
	Spring Security Spring Security es el módulo de seguridad del ecosistema Spring. Se emplea para proteger los endpoints del backend y aplicar control de acceso por roles, diferenciando usuarios como ADMIN, ANALYST y VIEWER. Aporta una capa centralizada de autenticación y autorización.

	JWT JWT, o JSON Web Token, es un estándar para transmitir información de identidad de forma firmada. Se utiliza para autenticar las peticiones realizadas al backend tras el inicio de sesión del usuario. El token permite identificar al usuario y sus roles en cada operación protegida. Aporta autenticación stateless y facilita la separación entre frontend y backend.
	PostgreSQL PostgreSQL es un sistema gestor de bases de datos relacional de código abierto. Se utiliza como base de datos principal para almacenar usuarios, roles, binarios, análisis, resultados, artefactos y eventos de auditoría. Aporta integridad, persistencia y capacidad para consultar el historial del sistema.
	JSONB JSONB es un tipo de dato de PostgreSQL que permite almacenar documentos JSON de forma optimizada. Se utiliza para guardar los resultados de análisis en la tabla analysis_results, ya que estos pueden variar según el tipo de análisis ejecutado. Aporta flexibilidad para almacenar resultados técnicos sin obligar a modificar el esquema relacional en cada evolución.

	MinIO / S3 MinIO es un sistema de almacenamiento de objetos compatible con la API S3 de AWS. Se utiliza para almacenar binarios subidos, informes HTML y artefactos generados por los análisis. La base de datos conserva únicamente las referencias a dichos objetos. Aporta separación entre metadatos y ficheros pesados, facilitando escalabilidad y mantenimiento.
	Docker Compose Docker Compose permite definir y ejecutar varios servicios mediante contenedores. Se utiliza para levantar la infraestructura local, principalmente PostgreSQL y MinIO. Aporta reproducibilidad del entorno y simplifica la puesta en marcha del sistema durante el desarrollo y las pruebas.
	React React es una librería de JavaScript para construir interfaces de usuario basadas en componentes. Se utiliza para desarrollar el frontend web, desde el cual el usuario puede autenticarse, subir binarios, lanzar análisis, consultar resultados, generar informes y revisar auditorías. Aporta una interfaz dinámica, modular y fácil de mantener.
	Vite Vite es una herramienta moderna para desarrollo y construcción de aplicaciones frontend. Se utiliza junto con React para ejecutar el entorno de desarrollo y generar la versión final del frontend. Aporta rapidez, recarga en caliente y un proceso de build ligero.

	Fetch nativo Fetch es la API nativa del navegador para realizar peticiones HTTP. Se utiliza mediante un cliente propio del frontend para comunicarse con el backend API. Aporta simplicidad, reduce dependencias externas y permite controlar de forma directa errores, cabeceras y credenciales.
	Python Python es un lenguaje interpretado muy utilizado en automatización, scripting y análisis técnico. Se utiliza para implementar los workers de análisis estático y dinámico. Estos componentes procesan los binarios, extraen señales, correlacionan resultados y calculan el scoring. Aporta flexibilidad y rapidez para desarrollar pipelines de análisis.
	C C es un lenguaje de bajo nivel que permite interactuar directamente con el sistema operativo. Se utiliza para desarrollar el agente de análisis dinámico, encargado de observar la ejecución del binario y emitir eventos estructurados. Aporta el control necesario para trabajar con mecanismos de instrumentación de bajo nivel.
	ptrace / seccomp ptrace permite observar y controlar la ejecución de un proceso en Linux. seccomp permite aplicar restricciones a las llamadas al sistema disponibles para un proceso. Se contemplan dentro del análisis dinámico controlado para observar la ejecución del binario y limitar su comportamiento en un entorno de laboratorio.

	Aportan capacidad de monitorización y control durante la fase dinámica del análisis.
	Flyway Flyway es una herramienta de migraciones de base de datos. Se utiliza para versionar y aplicar los cambios del esquema de PostgreSQL de forma ordenada. Aporta reproducibilidad y control sobre la evolución de la base de datos.
 	Git y GitHub Git es un sistema de control de versiones y GitHub una plataforma para alojar repositorios. Se ha limitado el uso de git en un repositorio privado local, y el MVP se ha subido a un repositorio de entrega final.

Tabla 18: Stack Tecnológico del Producto

7. METODOLOGÍA

7.1 Enfoque metodológico

Para el desarrollo se ha seguido una metodología iterativa e incremental, adecuada para un proyecto individual en el que el producto combina varias áreas técnicas: desarrollo backend, frontend web, persistencia, análisis de binarios, ejecución dinámica controlada, almacenamiento de artefactos, seguridad y auditoría.

Este enfoque permite dividir el proyecto en bloques funcionales pequeños, verificables y defendibles. Cada iteración se centra en entregar una parte completa del sistema, desde la base técnica hasta la integración con la interfaz y las pruebas. De esta forma, se reduce el riesgo de construir componentes aislados que no puedan integrarse posteriormente.

La metodología se ha organizado en torno a los requisitos R01–R08 definidos en el apartado de objetivos. Esta trazabilidad permite justificar qué parte del sistema responde a cada objetivo y cómo se valida su cumplimiento.

El proyecto se ha desarrollado con una visión de MVP evolutivo. En primer lugar, se prioriza la creación de una base funcional mínima: autenticación, persistencia, subida de binarios y modelo de datos. Posteriormente se incorporaron los módulos de análisis estático, análisis dinámico controlado, visualización de resultados, generación de informes y auditoría. Finalmente, se reforzó la calidad del producto mediante pruebas, documentación técnica, casos de uso y automatización local.

7.2 Justificación de la metodología

La elección de una metodología iterativa se justifica por las siguientes razones:

El proyecto integra tecnologías heterogéneas: Java/Spring Boot, React/Vite, PostgreSQL, MinIO/S3, Python y C.

El análisis de binarios requiere validación progresiva con muestras de prueba.

La arquitectura incluye varios componentes independientes que deben coordinarse correctamente: frontend web, backend API, workers de análisis, agente dinámico C, base de datos y almacenamiento de objetos.

El sistema necesita trazabilidad entre requisitos, implementación y pruebas.

Al tratarse de un proyecto individual, es necesario controlar el alcance y priorizar entregas verificables frente a ampliaciones excesivas.

La metodología adoptada ha permitido mantener una evolución controlada del proyecto, evitando introducir funcionalidades demasiado complejas y conservando el enfoque ético definido desde la propuesta inicial.

7.3 Fases de trabajo

Fase	Descripción	Requisitos relacionados	Resultado principal
Fase 1	Análisis inicial, definición del alcance y diseño funcional	R01–R08	Propuesta, alcance MVP y RFTP inicial
Fase 2	Diseño de arquitectura y modelo de datos	R01, R02, R03, R08	Arquitectura por componentes y esquema PostgreSQL
Fase 3	Implementación de autenticación, autorización y sesiones	R01	Login, registro, JWT, refresh token y RBAC
Fase 4	Persistencia, almacenamiento y subida de binarios	R02, R03	PostgreSQL, MinIO/S3, SHA-256 y deduplicación
Fase 5	Análisis estático ELF	R04	Worker estático, extracción ELF, señales y scoring
Fase 6	Análisis dinámico controlado	R05	Worker dinámico, agente C, timeout y trazas
Fase 7	Visualización de resultados e informes	R06, R07	UI de análisis, resultados JSONB e informes HTML
Fase 8	Auditoría y administración	R08	Eventos auditables, filtros y gestión administrativa
Fase 9	Pruebas, automatización y documentación final	R01–R08	Tests, smoke E2E, casos de uso y memoria final

Tabla 19: Fases de trabajo por requisito

7.4 Planificación inicial por requisitos

La siguiente tabla resume la planificación estimada del proyecto por bloques de requisitos. Las horas son estimativas y representan dedicación efectiva de análisis, diseño, implementación, pruebas y documentación.

Requisito	Bloque de trabajo	Fecha inicio	Fecha fin	Horas estimadas	Entregable
R01	Autenticación, autorización y roles	11/02/2026	16/02/2026	24 h	Registro, login, JWT, RBAC
R02	Persistencia y almacenamiento de resultados	16/02/2026	21/02/2026	24h	PostgreSQL, migraciones, JSONB
R03	Subida y gestión de binarios	21/02/2026	26/02/2026	22 h	Upload, SHA-256, MinIO/S3
R04	Análisis estático ELF	26/02/2026	01/03/2026	28 h	Worker estático y extracción ELF
R05	Análisis dinámico controlado	02/03/2026	11/03/2026	36 h	Worker dinámico, agente C y timeout
R06	Almacenamiento y visualización de resultados	12/03/2026	20/03/2026	28 h	Resultados JSONB y vista de análisis
R07	Generación y descarga de informes	21/03/2026	26/03/2026	20 h	Informe HTML y artefactos
R08	Auditoría de acciones	27/03/2026	03/04/2026	24 h	Eventos auditables y filtros
R09	Pruebas, documentación y cierre	04/04/2026	14/04/2026	34 h	Tests, smoke E2E, memoria y demo
	Total estimado			240 h	

Tabla 20: Planificación por horas

7.5 Planificación final y desviaciones

Durante el desarrollo, la planificación evolucionó respecto a la estimación inicial. La desviación principal se produjo porque la implementación real del MVP incorporó funcionalidades más completas que las previstas inicialmente. Entre ellas destacan la rotación de refresh token, la deduplicación por hash, el almacenamiento de artefactos en MinIO/S3, el contrato homogéneo de resultados results_json, la auditoría con filtros, la gestión administrativa de usuarios y la automatización de pruebas locales.

Área	Plan inicial	Resultado final	Desviación
Autenticación	Login y JWT básico	JWT, refresh token, logout y RBAC	Mejora de seguridad y sesión
Persistencia	PostgreSQL y JSONB	PostgreSQL, Flyway, JSONB, binary_uploads y refresh_sessions	Modelo más completo
Almacenamiento	Guardado de binarios	MinIO/S3 para binarios, informes y trazas	Mayor separación de responsabilidades
Análisis estático	ELF básico	Extracción ELF, mitigaciones, señales y correlación	Mayor profundidad técnica
Análisis dinámico	Timeout y syscalls básicas	Agente C, timeout, trazas, eventos y resumen dinámico	Mayor complejidad controlada
Resultados	Consulta básica	Vista por pestañas y contrato results_json v1	Mejor experiencia de usuario
Informes	HTML básico	Informe HTML estructurado con resumen y hallazgos	Mayor valor documental
Auditoría	Registro de eventos	Auditoría append-only con filtros y consulta ADMIN	Mejora de trazabilidad
Pruebas	Pruebas manuales	Tests backend, workers, build frontend y smoke E2E	Mayor reproducibilidad

Tabla 21: Desviaciones en base a propuesta inicial

7.6 Validación metodológica

El avance del proyecto se ha validado mediante pruebas incrementales asociadas a cada requisito. En las primeras fases se verificaron autenticación, autorización, conexión con base de datos y subida de binarios. Posteriormente se añadieron pruebas específicas para los workers de análisis, generación de informes y auditoría.

Además, se incorporó una automatización local mediante Makefile, que permite levantar el entorno, ejecutar pruebas y realizar un flujo smoke E2E. Este flujo cubre comprobaciones representativas del MVP: health check, registro/login, subida de ELF, análisis estático, análisis dinámico, generación/descarga de informe y consulta de auditoría filtrada.

Esta forma de validación refuerza la calidad del proyecto porque demuestra que los componentes no solo funcionan de forma aislada, sino también integrados dentro de un flujo completo de uso.

7.7 Diagrama de Gantt

Ruta en el proyecto: /docs/gantt_diagram.pdf/xlsx

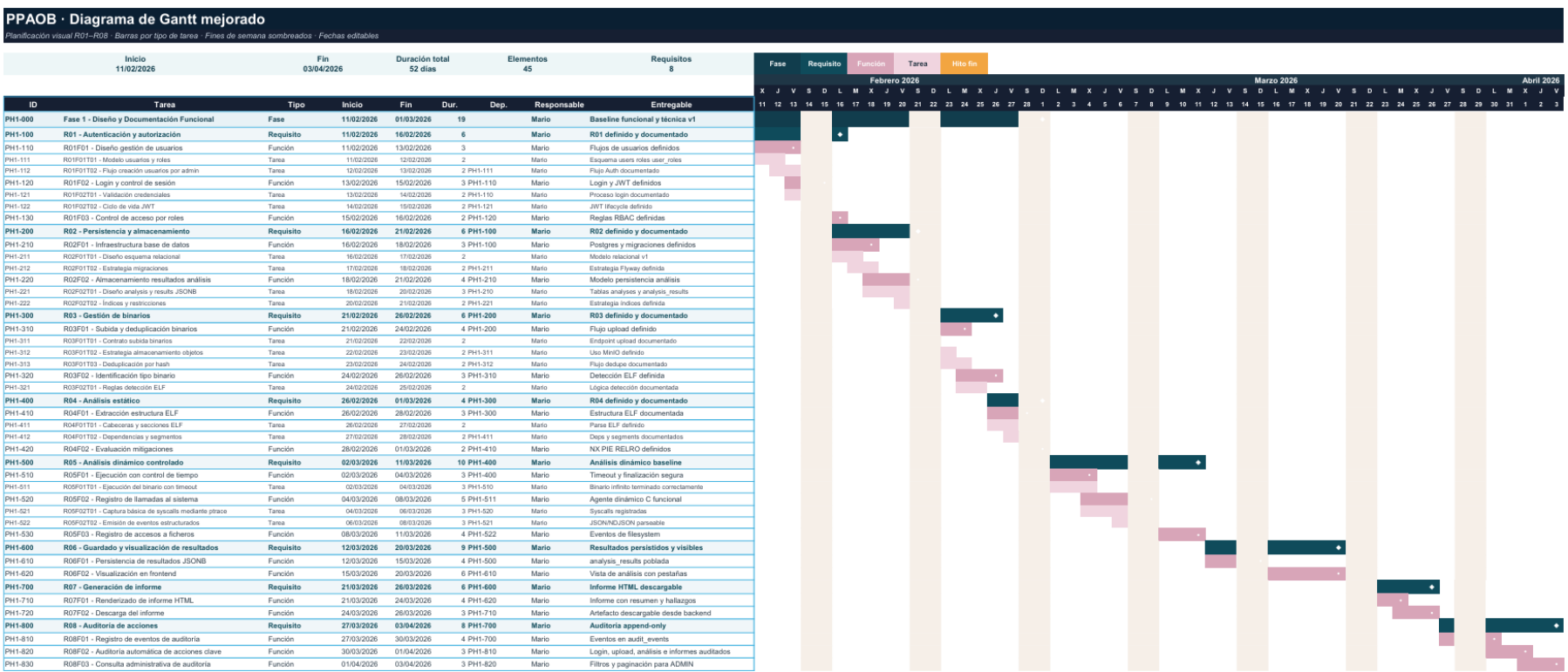


Ilustración 20: Diagrama de Gantt

8. PRESUPUESTO

Para calcular el coste se ha tomado como referencia una dedicación total estimada de **240 horas**, coherente con la planificación del proyecto. Se aplica un coste orientativo de **20 €/hora**, correspondiente a un perfil junior de desarrollo software con participación en tareas de backend, frontend, bases de datos, automatización y documentación técnica.

8.1 Coste de Desarrollo por Requisito

Requisito	Bloque de trabajo	Horas estimadas	Coste/hora	Coste total
R01	Autenticación, autorización y roles	24 h	20 €/h	480€
R02	Persistencia e historial de análisis	24 h	20 €/h	480€
R03	Subida y gestión de binarios	22 h	20 €/h	440€
R04	Análisis estático ELF	28 h	20 €/h	560€
R05	Análisis dinámico controlado	36 h	20 €/h	720€
R06	Almacenamiento y visualización de resultados	28 h	20 €/h	560€
R07	Generación y descarga de informes	20 h	20 €/h	400€
R08	Auditoría de acciones	24 h	20 €/h	480€
R09	Pruebas, documentación y cierre	34 h	20 €/h	680€
	Total desarrollo	240 h		4.800€

Tabla 22: Coste de desarrollo

8.2 Costes de infraestructura y herramientas

El proyecto se ha desarrollado principalmente con herramientas abiertas y servicios ejecutados en local. La infraestructura utilizada se basa en contenedores Docker, PostgreSQL y MinIO/S3 compatible, por lo que no se han requerido servicios cloud de pago para el MVP.

Concepto	Descripción	Coste estimado
Equipo de desarrollo	Uso amortizado de ordenador personal durante el proyecto	120€
Electricidad y conectividad	Consumo eléctrico y conexión a Internet durante el desarrollo	40€
PostgreSQL	Base de datos de código abierto usada en local	0€
MinIO	Almacenamiento de objetos compatible con S3 usado en local	0€
Docker / Docker Compose	Entorno de contenedores local	0€
Java / Spring Boot	Desarrollo backend	0€
React / Vite	Desarrollo frontend	0€
Python y C	Workers y agente dinámico	0€
Git / GitHub	Control de versiones y repositorio de entrega	0€
Herramientas de documentación	Documentación técnica y memoria	0€
	Total infraestructura y herramientas	160€

Tabla 23: Coste Infraestructura y Herramientas

8.3 Coste Total Estimado

Partida	Coste
Desarrollo por requisitos	4.800€
Infraestructura y herramientas	160€
Total estimado del proyecto	4.960€

Tabla 24: Coste total estimado

Por tanto, el coste estimado total del desarrollo de PPAOB asciende a **4.960 €**. Este importe refleja el valor aproximado del trabajo técnico realizado y de los recursos utilizados, aunque en el contexto académico del TFG no se haya producido una facturación real.

9. README y GIT.

Durante el desarrollo se ha mantenido una estructura de proyecto organizada y orientada a facilitar la revisión técnica. El repositorio incluye documentación de arquitectura, detalle de implementación, scripts de infraestructura, configuración de base de datos, frontend, backend, workers de análisis y agente dinámico.

El repositorio final sirve como punto de entrega del código fuente y de la documentación asociada. Además, el proyecto incluye ficheros de apoyo como README.md, documentación técnica y automatización mediante Makefile, lo que facilita la puesta en marcha y validación del sistema.

El README cumple una función complementaria a la memoria, ya que resume el propósito del proyecto, su estructura, las tecnologías utilizadas, el enfoque ético y las instrucciones básicas para comprender el repositorio. La memoria, por su parte, desarrolla la justificación académica, el diseño, la metodología, el presupuesto, las pruebas y las conclusiones.

Repositorio de entrega: [mario-asenjo/TFG-DAM-PPAOB](https://github.com/mario-asenjo/TFG-DAM-PPAOB)

10. TRABAJOS FUTUROS

Aunque el proyecto cumple con el alcance principal definido para el MVP, PPAOB se ha diseñado como una plataforma extensible. Por ello, existen varias líneas de mejora que podrían abordarse en futuras versiones.

Una primera ampliación sería incorporar soporte para otros formatos de binarios, especialmente **PE en Windows**. Actualmente el proyecto se centra en binarios ELF sobre Linux, por lo que añadir PE permitiría ampliar los escenarios de análisis y adaptar la plataforma a otros entornos.

Otra mejora relevante sería reforzar el **aislamiento del análisis dinámico**. En esta versión se contempla una ejecución controlada con trazado y límite de tiempo, pero futuras versiones podrían incorporar mecanismos de sandboxing más avanzados, perfiles de contención más estrictos y entornos de ejecución desechables.

11. CONCLUSIONES

A nivel técnico, el proyecto ha supuesto la integración de varias áreas de conocimiento: desarrollo backend con Spring Boot, frontend web con React, persistencia en PostgreSQL, almacenamiento de objetos con MinIO/S3, análisis de binarios mediante workers especializados y uso de un agente dinámico en C para observar la ejecución en un entorno controlado.

Uno de los aspectos más relevantes del sistema es la trazabilidad. La incorporación de autenticación, autorización por roles y auditoría de acciones permite saber quién realiza cada operación y cuándo, algo especialmente importante en entornos de análisis de seguridad. Además, la generación de informes HTML facilita la revisión posterior de los resultados y aporta valor documental al proceso de auditoría.

Durante el desarrollo también se han abordado retos importantes, como la separación entre backend y workers, la gestión de binarios mediante hash, la persistencia flexible de resultados con JSONB, la integración con almacenamiento S3 compatible y la ejecución dinámica controlada.

Como resultado, PPAOB demuestra que es posible construir una solución académica con visión de producto, capaz de organizar y priorizar información técnica en entornos de análisis ofensivo de binarios.

Me ha permitido consolidar conocimientos de desarrollo de aplicaciones, arquitectura software, bases de datos, seguridad y documentación técnica.

12. REFERENCIAS

1. Aplicado en la implementación del backend y la estructura general de la API REST.
Oracle. (s. f.). *Java Documentation*. Recuperado de <https://docs.oracle.com/en/java/>
2. Aplicado en el desarrollo del backend y la configuración de la aplicación principal.
VMware Tanzu. (s. f.). *Spring Boot Reference Documentation*. Recuperado de <https://docs.spring.io/spring-boot/documentation.html>
3. Aplicado en la autenticación, autorización y control de acceso por roles.
VMware Tanzu. (s. f.). *Spring Security Reference Documentation*. Recuperado de <https://docs.spring.io/spring-security/reference/index.html>
4. Aplicado en el sistema de autenticación basado en tokens JWT.
Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT), RFC 7519*. Internet Engineering Task Force. Recuperado de <https://datatracker.ietf.org/doc/html/rfc7519>
5. Aplicado en el diseño de la base de datos relacional y persistencia del sistema.
PostgreSQL Global Development Group. (s. f.). *PostgreSQL Documentation*. Recuperado de <https://www.postgresql.org/docs/>
6. Aplicado en el almacenamiento flexible de resultados de análisis mediante JSONB.
PostgreSQL Global Development Group. (s. f.). *JSON Types*. Recuperado de <https://www.postgresql.org/docs/current/datatype-json.html>
7. Aplicado en el almacenamiento de binarios, informes y artefactos mediante object storage compatible con S3.
MinIO, Inc. (s. f.). *MinIO Documentation*. Recuperado de <https://docs.min.io/>
8. Aplicado en la definición del entorno local reproducible mediante contenedores.
Docker Inc. (s. f.). *Docker Compose Documentation*. Recuperado de <https://docs.docker.com/compose/>
9. Aplicado en el desarrollo de la interfaz web basada en componentes.
Meta Open Source. (s. f.). *React Documentation*. Recuperado de <https://react.dev/>
10. Aplicado en el entorno de desarrollo y construcción del frontend.
Vite. (s. f.). *Vite Documentation*. Recuperado de <https://vite.dev/>

11. Aplicado en la comunicación HTTP entre frontend y backend mediante Fetch nativo.
Mozilla Developer Network. (s. f.). *Fetch API*. Recuperado de https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
12. Aplicado en la implementación de los workers de análisis estático y dinámico.
Python Software Foundation. (s. f.). *Python Documentation*. Recuperado de <https://docs.python.org/>
13. Aplicado en la gestión versionada de migraciones de base de datos.
Redgate. (s. f.). *Flyway Documentation*. Recuperado de <https://documentation.red-gate.com/fd>
14. Aplicado en el control de versiones del código fuente y documentación del proyecto.
Software Freedom Conservancy. (s. f.). *Git Documentation*. Recuperado de <https://git-scm.com/docs>
15. Aplicado en el análisis de binarios ELF sobre Linux.
Kerrisk, M. (s. f.). *elf(5) — Linux manual page*. Recuperado de <https://man7.org/linux/man-pages/man5/elf.5.html>
16. Aplicado en el agente de análisis dinámico y observación de procesos.
Kerrisk, M. (s. f.). *ptrace(2) — Linux manual page*. Recuperado de <https://man7.org/linux/man-pages/man2/ptrace.2.html>
17. Aplicado en el control de llamadas al sistema durante la ejecución dinámica controlada.
Kerrisk, M. (s. f.). *seccomp(2) — Linux manual page*. Recuperado de <https://man7.org/linux/man-pages/man2/seccomp.2.html>